МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

Разработка и анализ требований:

средства прототипирования

Учебно-методическое пособие

ГУАП

Санкт-Петербург

2022

Рецензенты:

кандидат технических наук, доцент *Т. Н. Костюнина*;

кандидат физ.-мат. наук, доцент *М. В. Фаттахова*

Богословская Н. В., Бржезовский А. В., Семененко Т. В.

Разработка и анализ требований: средства прототипирования: учебно-методическое пособие / Н. В. Богословская, А. В. Бржезовский, Т. В.Семененко. – СПБ: ГУАП 2022. – 70 с.

Содержание дисциплины «Теория информационных процессов и систем» охватывает круг вопросов, связанных с выявлением требований в проектах разработки информационных процессов и систем. В рамках дисциплины рассматриваются современные подходы к анализу и моделированию бизнес-процессов в нотации BPMN, спецификации требований, использованию прототипов в процессе разработки ПО.

В учебно-методическом пособии рассмотрены вопросы использования средств для быстрой разработки прототипа системы (процесса). В пособие включены описания основных средств, обеспечивающих создание базы данных, подключение приложения к СУБД и БД, выполнение запросов, конструирование интерфейса, представление данных в интерфейсе пользователя.

Описания средств, приведенные в пособии, могут рассматриваться как сведения начального уровня, необходимые для понимания основ создания слоя доступа к данным при изучении дисциплин «Управление данными», «Интерфейсы информационных систем» и курсового проектирования по дисциплине «Методы и средства проектирования информационных систем и технологий».

Примеры, представленные в пособии, реализованы на языке C# с применением СУБД Microsoft SQL Server.

Учебное пособие предназначено для студентов, обучающихся по направлению 09.03.02 — Информационные системы и технологии.

1. ОСНОВНЫЕ ПОНЯТИЯ ТЕОРИИ ИНФОРМАЦИОННЫХ ПРОЦЕССОВ И СИСТЕМ

Теория информационных процессов – совокупность моделей и методов, предназначенных для анализа информационных процессов, происходящих в технических, экономических, социальных, биологических, экологических и других системах.

Основные информационные процессы: формирование, преобразование, сбор, хранение, обработка, распределение, использование информации и др. Каждый информационный процесс характеризуется целевой функцией, реализуемой конкретными исполнителями или техническими средствами.

Процессы, обеспечивающие работу информационной системы любого назначения:

- ввод информации из внешних или внутренних источников;
- обработка входной информации и представление её в удобном виде;
- вывод информации для представления потребителям или передачи в другую систему;
- обратная связь – это информация, переработанная людьми данной организации для коррекции входной информации.

Для создания информационной системы или автоматизации информационного процесса необходимо разработать требования к системе (процессу). ***Требования*** — это спецификация того, что должно быть реализовано. В них описано поведение системы, свойства системы, ограничения в работе. Требования описывают представление пользователя, внешнее поведение системы, а также представление разработчика о некоторых внутренних характеристиках [1].

Одним из важных этапов разработки требований является этап прототипирования.

Прототип – реализация основных функциональных возможностей разрабатываемой системы (процесса) для обсуждения с заказчиками и пользователями. Прототип позволяет выполнить анализ работы системы (процесса), оценить эффективность, выявить ошибки, критические ситуации. Прототип является частью этапа анализа и разработки требований и становится приложением к техническому заданию.

Этап прототипирования решает задачу проверки корректности проекта, является основой для этапа тестирования [1].

Прототипирование не обязательно выполняется в рамках тех же технологий, что и разрабатываемая система. В учебно-методическом пособии рассмотрены возможные технологии и приемы разработки прототипов информационных процессов и систем.

1.1 Централизация хранения данных в информационных системах

Информационные процессы и системы обычно работают с данными. Независимо от форматов представления данных, функциональность системы создается для ввода, изменения, отображения, удаления, обработки и использования данных. Бизнес-аналитик должен начинать собирать определения данных по мере того, как они появляются в процессе выявления требований. Требования к данным охватывают все уровни разработки системы.

Анализ, моделирование и разработка информационных процессов и систем, как правило, выполняется с помощью различных программных инструментов. В практических примерах методических указаний все модели будут выполнены в Case-инструментарии SAP Sybase PowerDesigner 16.5. Программный код приложений реализован в MS Visual Studio 2019. Для размещения базы данных используется MS SQL Sever 2014.

В примерах пособия рассмотрены приемы анализа и представления данных, которые важны для пользователей разрабатываемой системы, а также способы создания таблиц базы данных и размещения их на сервере баз данных.

Технология моделирования данных представлена моделями «сущность-связь», предложенными Питером Пин-Шен Ченом (англ. Peter Pin-Shen Chen) [2].

1.2 Разработка концептуальной модели

Нормализация данных – это процесс приведения модели к виду, позволяющему получить в дальнейшем структуру базы данных, в которой устранена избыточность хранения и сведены к минимуму аномалии при добавлении, удалении, изменении данных. В процессе нормализации модель должна быть последовательно приведена к первой, второй, третьей нормальным формам (1НФ, 2НФ, 3НФ).

Приведение к 1НФ состоит в исключении множественных или повторяющихся атрибутов.

Атрибут (группа атрибутов) является множественным, если для одного экземпляра сущности может быть несколько различных значений атрибута. В примере на рисунке 1.1 таким атрибутом является «Оценка», так как студент имеет много оценок по различным предметам.



Рисунок 1.1 - Пример множественного (повторяющегося) атрибута

Приведение к 1НФ осуществляется по следующим правилам (рисунок 1.2):

- множественный атрибут или группа таких атрибутов выделяются в самостоятельную сущность;
- между исходной и новой сущностями устанавливается связь.

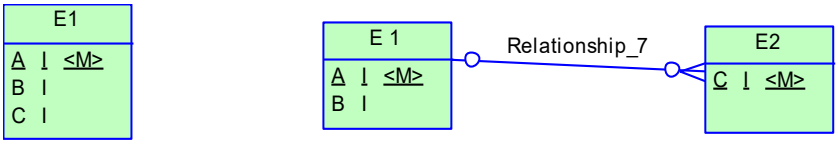


Рисунок 1.2 - Правило приведения к 1НФ

Результат приведения к 1НФ для сущности «СТУДЕНТ» (рисунок 1.1) показан на рисунке 1.3.

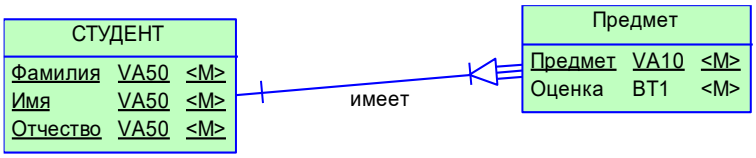


Рисунок 1.3 - Приведение к 1НФ сущности Студент

Первая нормальная форма, таким образом, является механизмом выявления упущенных сущностей и связей.

Приведение ко 2НФ предполагает вынесение атрибутов, которые не удовлетворяют требованиям функционально полной зависимости от уникального идентификатора сущности, являющегося составным ключом.

Атрибут А функционально зависит от атрибута В (ФЗ : В ⟶ А), если каждому значению А соответствует не более одного значения В. Функциональная зависимость может иметь место не только от одного атрибута, но и от группы атрибутов – ФЗ : В, С, D ⟶ А.

Функциональная зависимость является полной, если атрибут зависит от составного ключа и не зависит ни от какого подмножества данного ключа, т. е. имеет место зависимость ФЗ : К1, К2, ..., Кn ⟶ А и не справедлива ни одна из следующих зависимостей: ФЗ : К1 ⟶ А; ФЗ : К2 ⟶ А; ... ФЗ : К1, К2 ⟶ А и т. д.

Пример функционально неполной зависимости приведен на рисунке 1.4. Здесь атрибуты «Факультет», «Курс» зависят от ключевого атрибута «Номер группы» и не зависят от других ключевых атрибутов: «Фамилия», «Имя», «Отчество».



Рисунок 1.4 - Пример функционально неполной зависимости

Приведение к 2НФ осуществляется по следующим правилам (рисунок 1.5):

- атрибуты, зависящие от части составного ключа, и сама эта часть выносятся в отдельную (новую) сущность и исключаются из исходной;
- ключом новой сущности становится часть ключа исходной, от которой имеет место функционально неполная зависимость;
- между исходной и новой сущностями устанавливается связь типа M : 1, ключевая со стороны «многие».

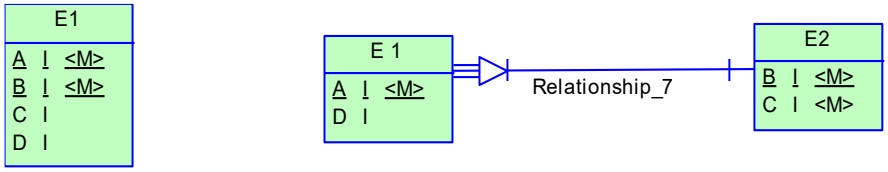


Рисунок 1.5 - Правило приведения ко 2НФ

Результат приведения ко 2НФ сущности “СТУДЕНТ” (рисунок 4) показан на рисунке 1.6.

Приведение к 2НФ, так же, как и приведение к 1НФ, позволяет выявить в модели упущенные сущности и связи и, кроме того, устранить избыточность данных в исходной модели.

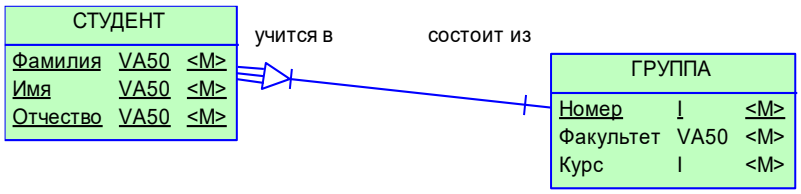


Рисунок 1.6 - Приведение ко 2НФ

В примере, рассматриваемом на рисунке 1.4, значение атрибутов «Факультет» и «Курс» повторялось бы для каждого экземпляра сущности «СТУДЕНТ», а, следовательно, неоправданно увеличивался бы объем памяти, занимаемой базой данных. Этот недостаток устранен в модели на рисунке 1.6. Кроме того, изменение, например, номера курса (рисунок 1.4), приводило бы к необходимости изменить значение атрибута для каждого экземпляра сущности «СТУДЕНТ», вместо того чтобы сделать это один раз для экземпляра сущности «ГРУППА» (аномалия изменения).

Приведение к 3НФ состоит в исключении транзитивных зависимостей атрибутов от атрибутов, не являющихся частью ключа.

Атрибут A транзитивно зависит от C, если A зависит от B, а B зависит от C (ФЗ : C —›* A, если ФЗ : B —› A и ФЗ : C —› B).

Пример транзитивной зависимости приведен на рисунке 1.7. Здесь значение атрибута «Должностной оклад» определяется неключевым атрибутом «Должность».



Рисунок 1.7 - Пример транзитивной зависимости

Приведение к 3НФ осуществляется по следующим правилам (рисунок 1.8):

- атрибуты, зависящие от не ключевых атрибутов, и сами эти атрибуты выносятся в отдельную (новую) сущность и исключаются из исходной сущности;
- ключом новой сущности становятся те атрибуты исходной, от которых была обнаружена транзитивная зависимость;
- между исходной и новой сущностями устанавливается связь типа M : 1.

Результат приведения к 3НФ сущности «СОТРУДНИК» (рисунок 1.7) показан на рисунке 1.9.

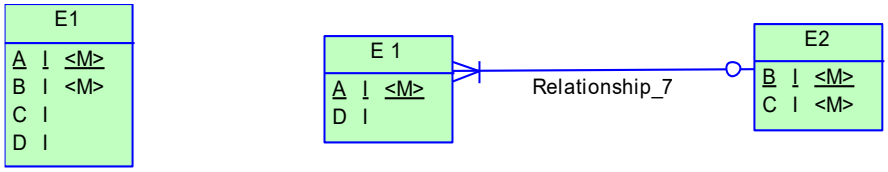


Рисунок 1.8 - Правила приведение к 3НФ

Приведение к 3НФ, так же, как и приведение к 1НФ, 2НФ, позволяет выявить в модели упущенные сущности и связи и, кроме того, устранить избыточность данных и аномалии изменения.

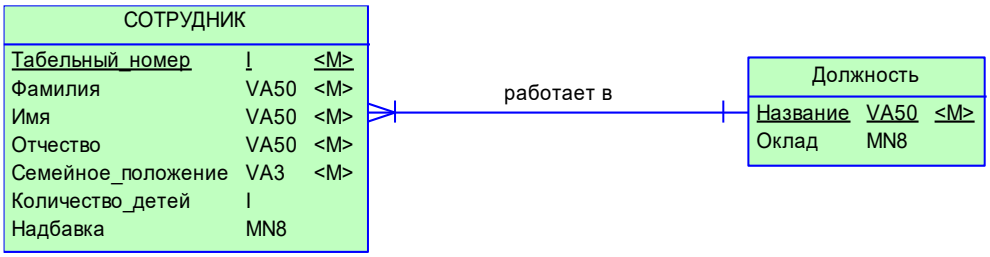


Рисунок 1.9 - Приведение к 3НФ

1.3 Технология разработки

Разработайте концептуальную модель вашего варианта, например, в CASE – редакторе PowerDesigner. Варианты описания автоматизированных процессов или систем для самостоятельной работы представлены в Приложении.

При создании новой модели следует выбрать шаблон построения концептуальной модели, как показано на рисунке 1.10.

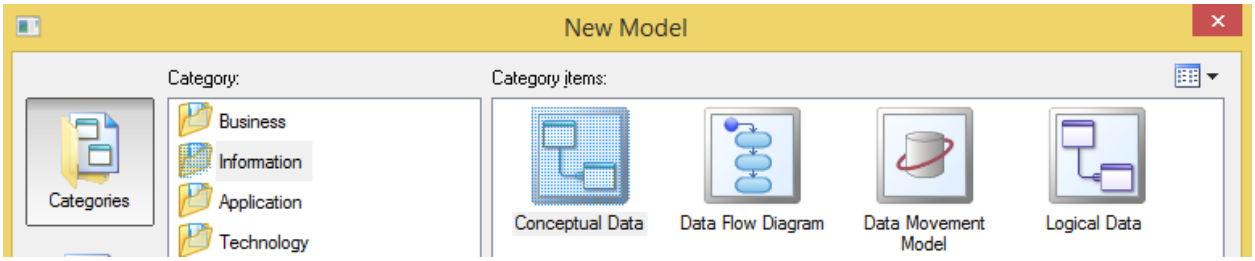


Рисунок 1.10 – Шаблоны построения модели

На рисунке 1.11 приведен пример концептуальной модели.

Выполните проверку модели Tools – Check Model. В случае успешной проверки запустите процесс создания физической структуры базы данных: Tools – Generate Physical Data Model.

Создайте сценарий для формирования таблиц Database – Generate Database.

После создания сценария должно появиться диалоговое окно Generated Files, в котором можно перейти в режим просмотра сценария в Блокноте с помощью кнопки Edit.

Сохраните сгенерированный сценарий в файле или скопируйте в буфер обмена.

Откройте SQL Server Management Studio. Создайте новую базу данных.

Перейдите в режим Ouery, откройте из файла или скопируйте сценарий из буфера обмена в окно редактора запросов и запустите сценарий на исполнение (рисунок 1.12).

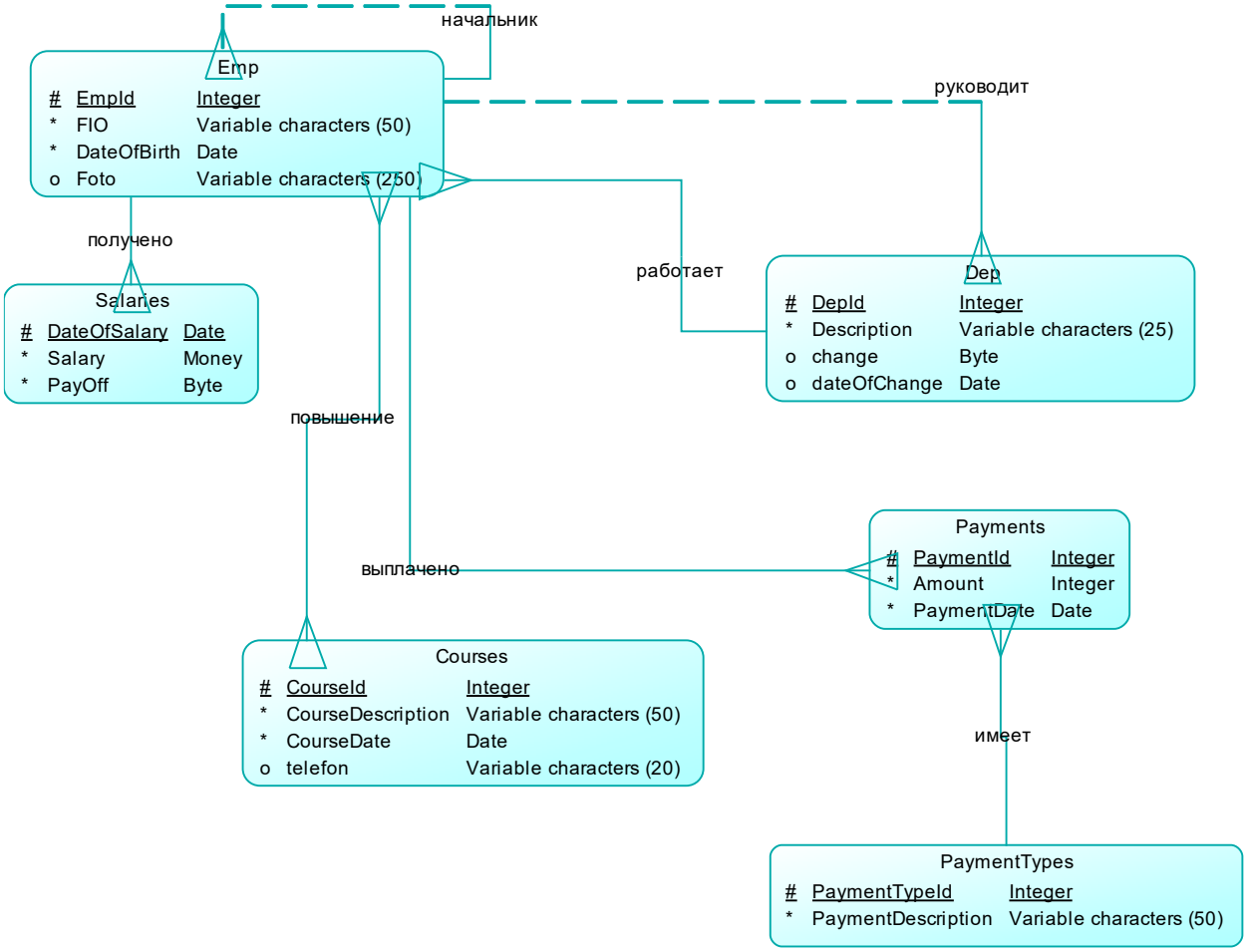


Рисунок 1.11 - Пример концептуальной модели

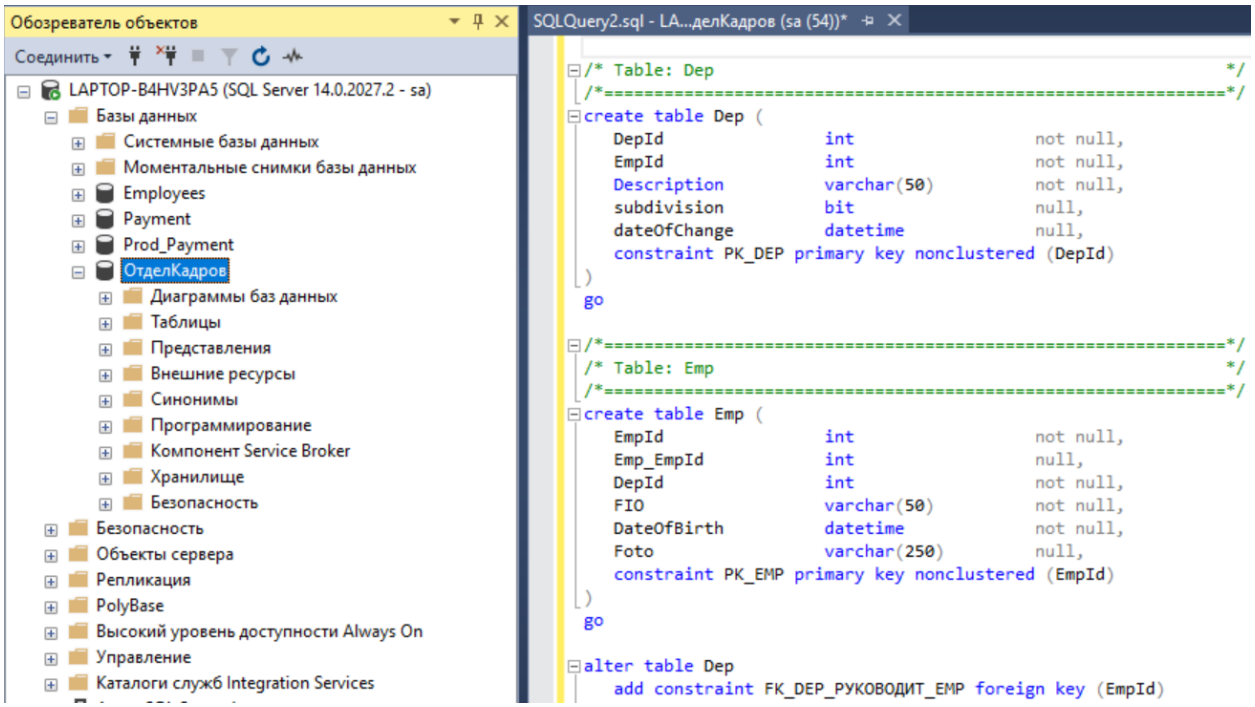


Рисунок 1.12 - Выполнение сценария для создания таблиц базы данных

Добавьте в структуру базы данных диаграмму (рисунок 1.13).

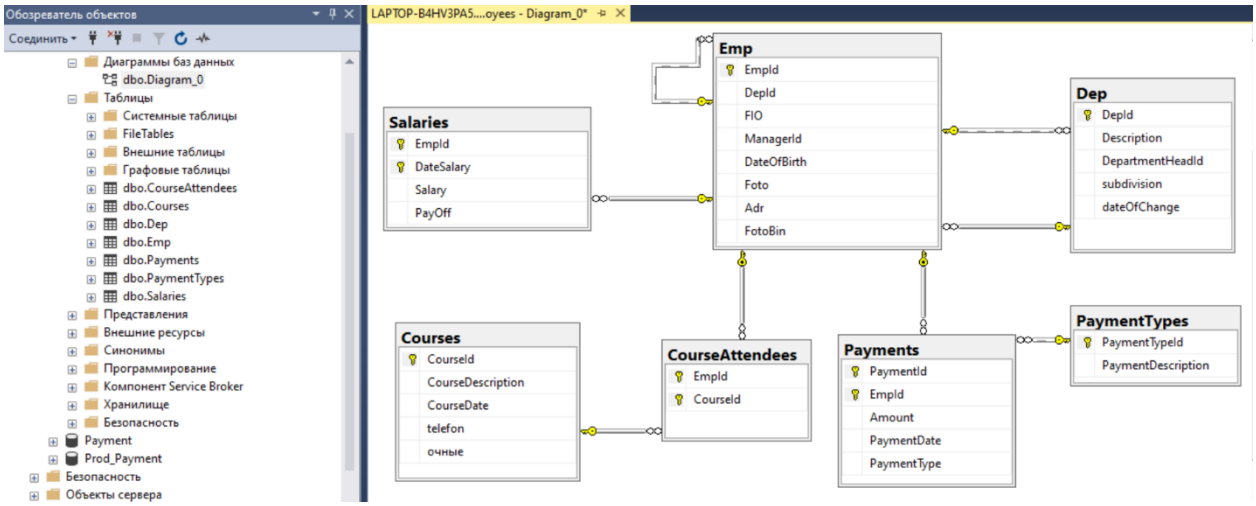


Рисунок 1.13 - Диаграмма таблиц и связей

2. ТЕХНОЛОГИИ ДОСТУПА К ДАННЫМ В ИНФОРМАЦИОННЫХ СИСТЕМАХ

ADO.NET — это набор классов, предоставляющих службы доступа к данным. ADO.NET удовлетворяет различные потребности разработчиков, включая создание клиентских приложений баз данных, а также бизнес-объектов среднего уровня, используемых приложениями, средствами, языками и браузерам [3].

2.1 Объект соединения

Объект ***Connection*** служит для установления соединения с заданными источниками данных. Соединение можно настроить, редактируя нужным образом значения параметров строки подключения - свойство ***ConnectionString***.

Базовый формат строки подключения содержит ряд пар параметров и значений, разделенных точкой с запятой. В следующем примере показана типичная строка подключения (https://msdn.microsoft.com/ ru-ru/library/system.data.sqlclient.sqlconnection.connectionstring(v=vs.110).aspx).

cnn.ConnectionString = "Persist Security Info=False; Integrated Security=true; Initial Catalog=Northwind; server=(local)";

Полный список параметров строки соединения и особенности их описания, например, указания сетевого адреса экземпляра SQL Server к которому выполняется подключение, приводятся в материалах для разработчиков компании Microsoft msdn.microsoft.com [3].

Объект Connection имеет методы Open() и Close(), которые открывают/закрывают подключение к базе данных со значениями свойств, описанными в свойстве ConnectionString.

Объект Connection имеет набор свойств, например, ServerVersion, State, которые отображают состояние объекта во время последней сетевой операции, выполненной при подключении.

На рисунке 2.1 приведен пример выполнения действий с объектом соединения для описания параметров соединения, открытия соединения, получения свойств соединения и закрытия соединения с базой данных MS SQL Server.

```
    //Создание объекта соединения
    SqlConnection cnn = new SqlConnection();

    //Вариант 1 создания объекта соединения
    private void button1_Click(object sender, EventArgs e)
    {
cnn = new SqlConnection(@"Data Source=ASUS;Initial Catalog=Employees;Integrated Security=True");
        // Открыть соединение
        cnn.Open();
        richTextBox1.Text = String.Format("Версия сервера:{0} \n", cnn.ServerVersion);
        richTextBox1.Text += String.Format("Состояние соединения: {0}\n",cnn.State.ToString());
        // Закрыть соединение
        cnn.Close();
        richTextBox1.Text += String.Format("Состояние соединения:{0} ", cnn.State.ToString());
    }
```

Рисунок 2.1 – Пример формирования объекта соединения

2.2 Управление строками соединений в windowsForms

При использовании различных поставщиков описание строк соединения удобнее выполнять в разделе <connectionStrings> конфигурационного файла приложения ***App.config***. На рисунке 2.2 приведен пример строки подключений к источнику данных, размещенной в области connectionStrings. Таких строк подключения к различным источникам может быть несколько; каждая строка должна иметь имя, которое может быть произвольным, но уникальным.

Еще одним параметром строки является providerName. Если вы используете базу данных, отличную от SQL Server, то необходимо познакомиться с документацией поставщика данных (или справочником по библиотеке классов .NET Framework), чтобы узнать о поддерживаемых значениях строки соединения.

Form1.cs | App.config* | Form1.cs [Конструктор]

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
    <startup>
        <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.6.1" />
    </startup>
  <connectionStrings>
    <add name="Employees" connectionString="Data Source=ASUS;Initial Catalog=Employees;
         Integrated Security=True" providerName="System.Data.SqlClient"/>
  </connectionStrings>
</configuration>
```

Рисунок 2.2 – Пример определения строки соединения в конфигураторе проекта

После того, как информация подключения к базе данных определена в конфигурационном файле, ее можно использовать в приложении. На рисунке 2.3 приведен программный код варианта создания объекта соединения, параметры которого прочитаны из конфигурационного файла при обращении к именованной области.

Для доступности класса ConfigurationManager следует добавить ссылки на сборку ***System.Configuration*** (https://docs.microsoft.com/ru-ru/dotnet/api/system.configuration.configurationmanager?view=netcore-2.2).

```
//Вариант 2 создания объекта соединения
private void button2_Click(object sender, EventArgs e)
{ // Получить параметры объекта Connection
    string connectionString = ConfigurationManager.ConnectionStrings["Employees"].ConnectionString;
    cnn = new SqlConnection(connectionString);
    // Открыть соединение
    cnn.Open();
    richTextBox1.Text += String.Format("Версия сервера:{0} \n", cnn.ServerVersion);
    richTextBox1.Text += String.Format("Состояние соединения: {0}\n", cnn.State.ToString());
    // Закрыть соединение
    cnn.Close();
    richTextBox1.Text += String.Format("Состояние соединения:{0} ", cnn.State.ToString());
}
```

Рисунок 2.3 – Получение параметров объекта соединения из именованной области App.config

2.3 Управление строками соединений в webForms

Технология ADO.NET унифицирует код доступа к данным для приложений любого типа. Создайте веб-приложение ***ASP.NET*** (.NET Framework) [4]. Для изучения архитектуры ADO.NET в контексте веб-приложений ASP.NET можно использовать самый простой шаблон веб-проекта как показано на рисунке 2.4.

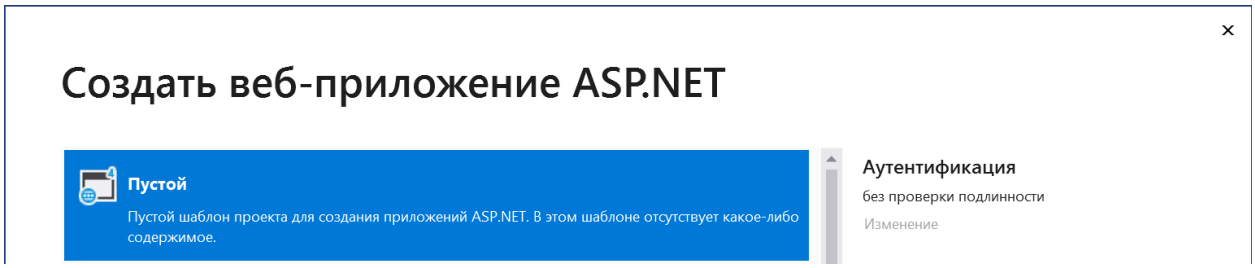


Рисунок 2.4 – Выбор пустого шаблона для исследования

Добавьте в проект Web-форму, разместите на странице элементы управления TextBox и Button. У элемента TextBox можно установить свойство TextMode в значение MultiLine. Рисунок 2.5 показывает структуру страницы с элементами интерфейса.

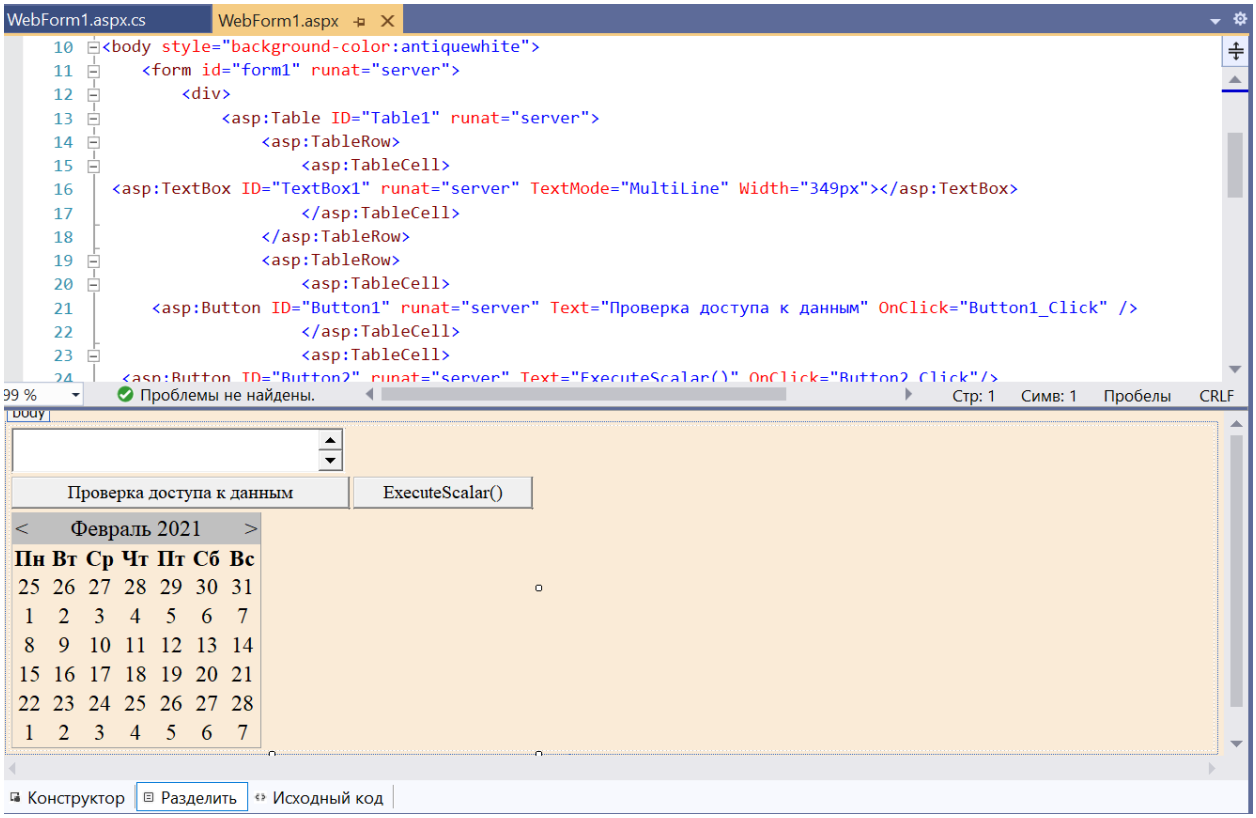


Рисунок 2.5 – Формирование интерфейса web-страницы

Можно скопировать разработанный ранее раздел connectionString в конфигурационный файл веб-приложения как на рисунке 2.6.

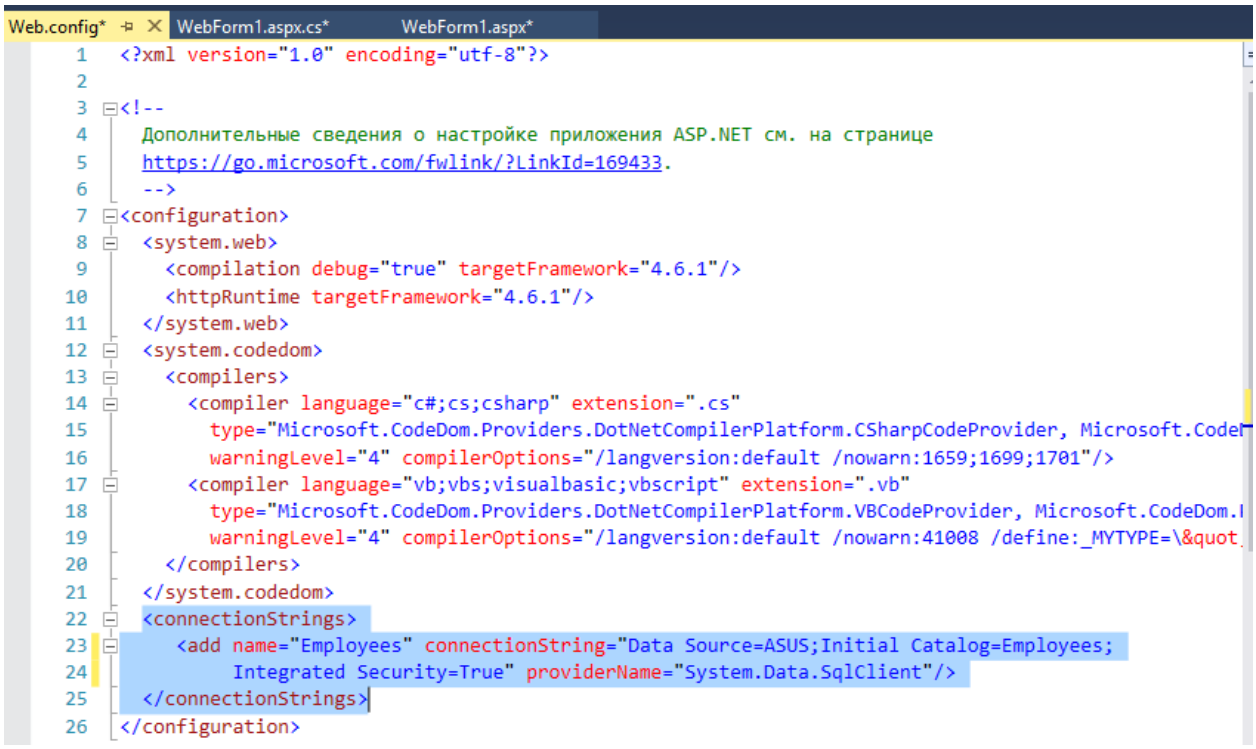


Рисунок 2.6 – Пример определения строки соединения в webForms

Эту строку соединения можно извлекать по имени из коллекции WebConfigurationManager.ConnectionStrings как показано на рисунке 2.7.

```
public partial class WebForm1 : System.Web.UI.Page
{
    string connectionString = ConfigurationManager.ConnectionStrings["Employees"].ConnectionString;
    SqlConnection cnn = new SqlConnection();
Ссылок: 0
protected void Button1_Click(object sender, EventArgs e)
    {
        cnn = new SqlConnection(connectionString);
        cnn.Open(); // Открыть соединение
        TextBox1.Text += String.Format("Версия сервера:{0} \n", cnn.ServerVersion);
        TextBox1.Text += String.Format("Состояние соединения: {0}\n", cnn.State.ToString());
        cnn.Close();// Закрыть соединение
        TextBox1.Text += String.Format("Состояние соединения:{0} ", cnn.State.ToString());
        TextBox1.Text += String.Format(new string('*', 42) + "\n");
    }
}
```

Рисунок 2.7 – Получение параметров объекта соединения из именованной области Web.config

2.4 Основные объекты провайдера данных: объект Command

Объект ***Command*** предоставляет методы, которые можно использовать для выполнения команды, в зависимости от того, нужно ли извлечь полный результирующий набор, получить единственное значение или просто

выполнить отличную от запроса команду. Рассмотрим использование этих методов в windows-приложении. Синтаксис языка SQL рассмотрен в [5].

Метод ***ExecuteScalar()*** выполняет команду SQL и возвращает первое поле первой найденной записи. Примерами использования метода являются команды SQL для вычисления суммы, среднего значения или общего количества записей (Sum, Avg, Count, Min, Max). Другими примерами являются справочные таблицы для подстановки одного значения или команды, возвращающие логические значения. Метод ExecuteScalar() выполняет заданную команду и возвращает значение только первого поля первой записи, остальные записи игнорирует.

Рассмотрим следующий пример команды: требуется определить количество сотрудников, которым была выплачена заработная плата 20 января 2021 года. В базе данных Employees необходимо выполнить следующую команду:

select count(EmpId) from Salaries where Salaries.DateSalary='2021-01-20'

Если для выполнения этой команды использовать метод ExecuteScalar(), то результатом будет единственное возвращаемое командой значение count(EmpId). На рисунке 2.8 показана таблица, к которой будет выполнен запрос. На рисунках 2.9 и 2.10 приведены, соответственно, интерфейсы формы и веб-страницы, на рисунке 2.11 представлен программный код для выполнения команды ExecuteScalar().

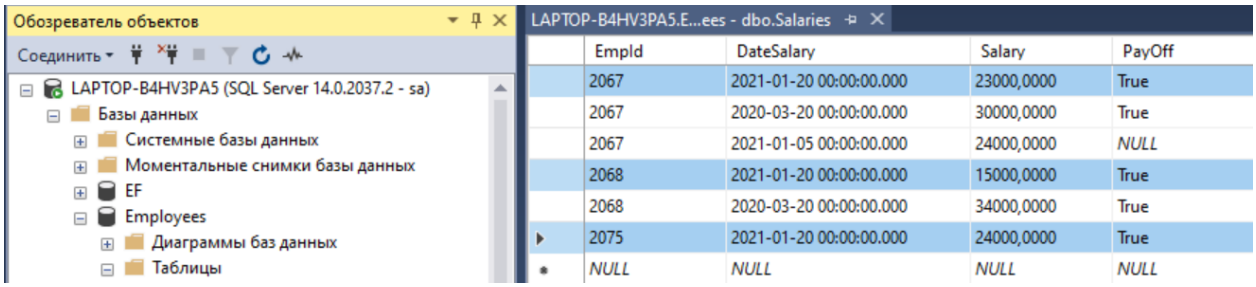


Рисунок 2.8 – Записи в таблице Salaries

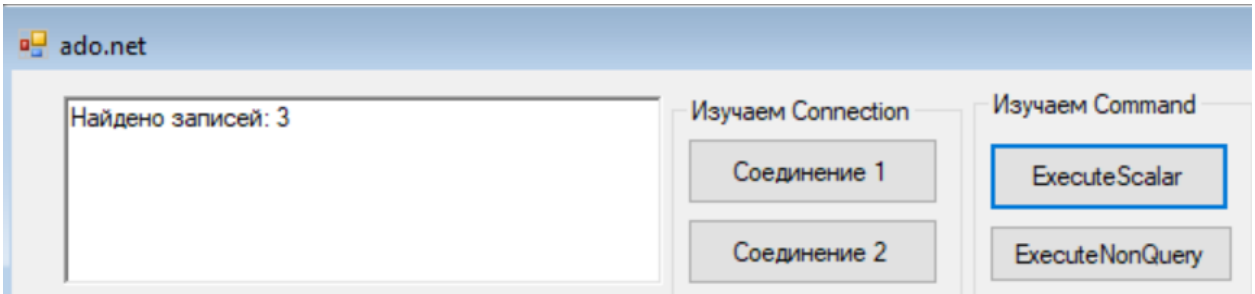


Рисунок 2.9 – Результат выполнения команды ExecuteScalar() в win-форме

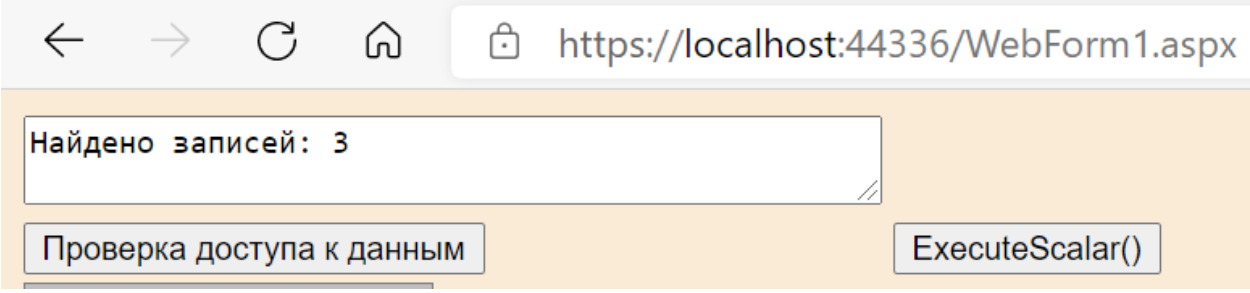


Рисунок 2.10 - Результат выполнения команды ExecuteScalar() в web-форме

```
  SqlCommand cmd = new SqlCommand();
//Метод ExecuteScalar()
ссылка: 1
private void button4_Click(object sender, EventArgs e)
{
    cnn = new SqlConnection(connectionString);
    // Открыть соединение
    cnn.Open();
    //Описать свойство Connection
    cmd.Connection = cnn;
    //Значение Text является значением по умолчанию, можно не описывать
    cmd.CommandType = System.Data.CommandType.Text;
    //Вычисляем количество сотрудников, которые получили зарплату в конкретную дату
     cmd.CommandText = "select count(EmpId) from Salaries where Salaries.DateSalary='2021-01-20'";
    //Выводим результат выполнения команды в интерфейс приложения
    richTextBox1.Text = String.Format("Найдено записей: {0} \n", cmd.ExecuteScalar());
    // Закрыть соединение
    cnn.Close();
}
```

Рисунок 2.11 – Программный вызов метода ExecuteScalar() в приложении WindowsForms

Метод ***ExecuteNonQuery()*** выполняет инструкцию Transact-SQL для установленного соединения и возвращает количество задействованных в инструкции строк.

При выполнении команд UPDATE, INSERT и DELETE метод ExecuteNonQuery() возвращают количество строк, которые были обработаны с их помощью. Для всех прочих типов команд возвращаемым значением является -1.

Рассмотрим следующую задачу: добавить описание нового курса для повышения квалификации сотрудников в таблицу Courses. На рисунке 2.12 приведен программный код команды Insert.

```
//Метод ExecuteNonQuery()
ссылка: 1
private void button5_Click(object sender, EventArgs e)
{
    cnn = new SqlConnection(connectionString);
    cnn.Open();
    cmd.Connection = cnn;
    //Вставляем новую строку в таблицу Courses
    cmd.CommandText = "insert into Courses values(1, 'Изучение ADO.NET','2021-03-05', '+7(921)678-90-89',1)";
    //Выполнить команду и показать результат
    richTextBox1.Text = String.Format("Обработано записей: {0} \n", cmd.ExecuteNonQuery());
    cnn.Close();
}
```

Рисунок 2.12 – Команда Insert для таблицы Courses

Метод ***ExecuteReader()*** выполняет запрос SELECT и возвращает объект DataReader.

Класс DataReader позволяет читать данные, возвращенные командой SELECT, по одной строке, в однонаправленном, доступном только для чтения потоке. Использование DataReader представляет наиболее быстрый способ доступа к данным.

Одним из основных методов DataReader является метод Read(). Метод перемещает курсор строки на следующую строку в потоке. Метод Read() возвращает true, если существует следующая строка для чтения, или false, если прочитана последняя строка в наборе.

Решим задачу просмотра списка сотрудников с указанием даты рождения. Из таблицы Emp по команде SELECT можно получить массив данных и прочитать его в приложении. На рисунке 2.13 показан результат запроса к таблице. На рисунке 2.14 приведен программный код для реализации.

Команда SQL может быть набором команд, например, такой случай рассмотрен на рисунке 2.15: сначала составляется список сотрудников с указанием даты рождения, а затем – список сотрудников с названием отделов, в которых они работают.

На рисунке 2.13 демонстрируется последовательное чтение полученных массивов данных с применением метода ***NextResult().***

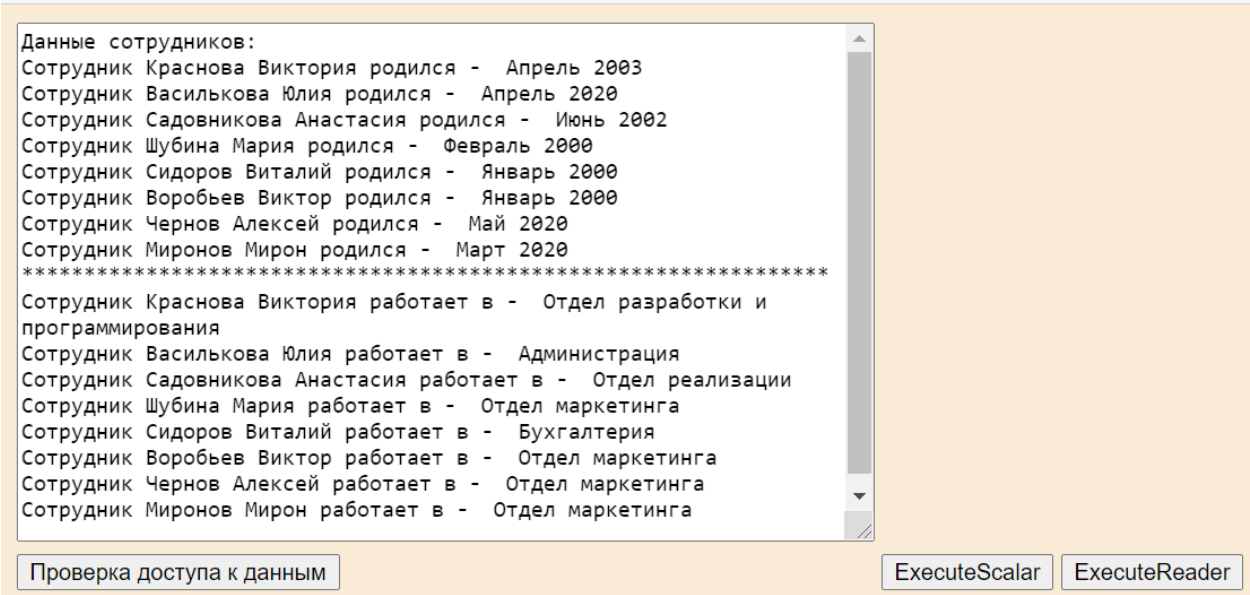


Рисунок 2.13 – Чтение двух наборов данных объектом DataReader в web-странице

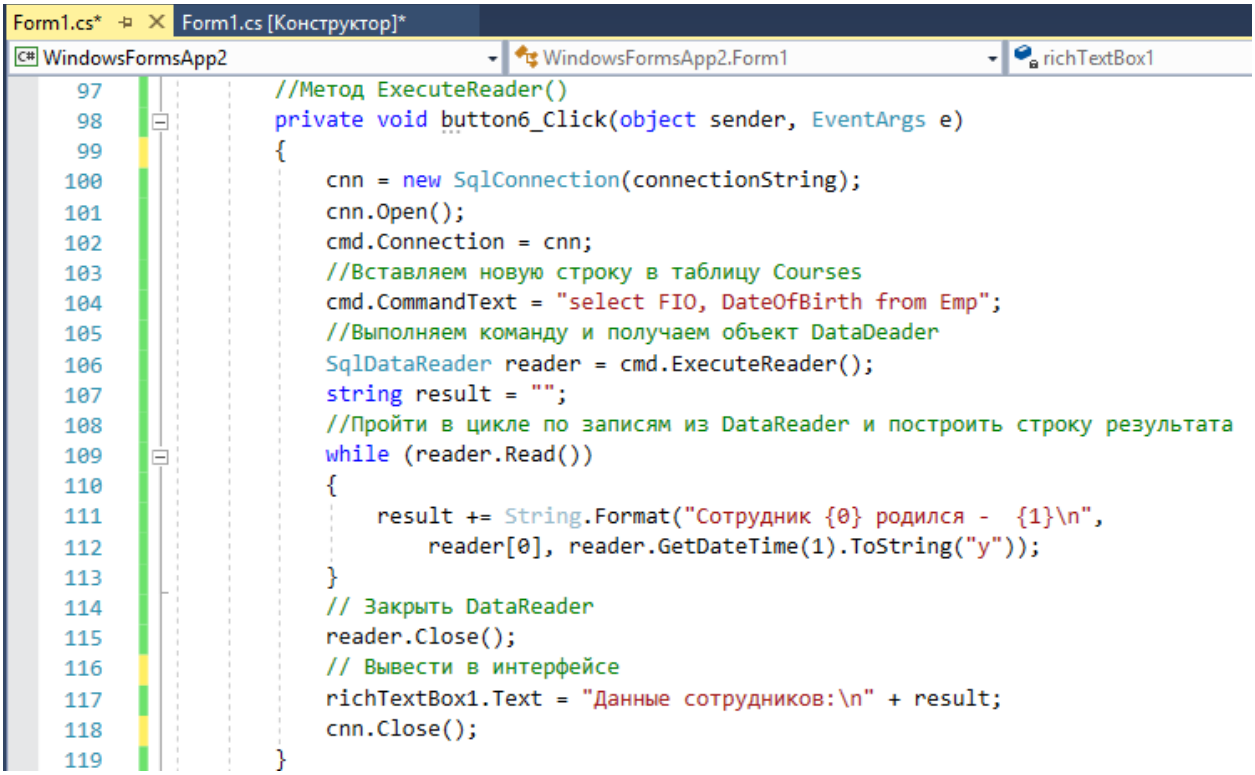


Рисунок 2.14 – Код win-формы для чтения небуферизуемого набора данных с помощью DataReader

```
//Метод ExecuteReader()
private void button6_Click(object sender, EventArgs e)
{
    cnn = new SqlConnection(connectionString);
    cnn.Open();
    cmd.Connection = cnn;
    //Вставляем новую строку в таблицу Courses
    cmd.CommandText = "select FIO, DateOfBirth from Emp; select FIO, Description from Emp,
      Dep  where Emp.DepId=Dep.DepId;";
    //Выполняем команду и получаем объект DataDeader
    SqlDataReader reader = cmd.ExecuteReader();
    string result = "";
    //Пройти в цикле по записям из DataReader и построить строку результата
    while (reader.Read())
    {
        result += String.Format("Сотрудник {0} родился -  {1}\n",
            reader[0], reader.GetDateTime(1).ToString("y"));
    }
    result += new string('*', 67);
    //Чтение следующего набора данных
    reader.NextResult();
    while (reader.Read())
    {
        result += String.Format("Сотрудник {0} работает в -  {1}\n",
            reader[0], reader[1]);
    }
    // Закрыть DataReader
    reader.Close();
    // Вывести в интерфейсе
    richTextBox1.Text = "Данные сотрудников:\n" + result;
    cnn.Close();
}
```

Рисунок 2.15 – Чтение нескольких наборов данных объекта DataReader

Методы ExecuteScalar(), ExecuteNonQuery(), ExecuteReader() имеют версии для асинхронного выполнения команд: пример на рисунке 2.16.

```
//Выполнение асинхронной команды и ввод параметров из интерфейса
ссылка: 1
async private void button8_Click(object sender, EventArgs e)
{
    cnn.ConnectionString = connectionString;
    cmd.Connection = cnn;
    cmd.CommandText="insert into Dep(DepId,Description) values("+Convert.ToInt16
      (textBox1.Text)+", '"+textBox2.Text+"')";
    await cnn.OpenAsync();
    int i=await cmd.ExecuteNonQueryAsync();
    cnn.Close();
    richTextBox1.Text = i.ToString();

}
```

Рисунок 2.16 – Реализация асинхронной команды и передачи параметров из интерфейса

3. ХРАНИМЫЕ ПРОЦЕДУРЫ

Хранимая процедура представляет собой откомпилированный набор инструкций SQL, хранимый под указанным именем и обрабатываемый как единое целое [5].

Хранимые процедуры вызываются клиентской программой, другой хранимой процедурой или триггером.

3.1 Синтаксис параметризованных команд

Параметризованная команда — это команда, которая использует символы-заполнители в тексте команды SQL. Заполнитель указывает место для динамически применяемых значений, которые затем передаются через коллекцию Parameters объекту Command.

Например, для определения количества сотрудников, которым была выплачена заработная плата 20 января 2021 года, запрос может быть следующим:

```
select count(EmpId) from Salaries
where Salaries.DateSalary='2021-01-20'
```

Очевидно, что передача параметру фактического значения в тексте SQL-команды – не самый удачный способ. Эффективнее использовать именованный параметр, описание которого можно добавить в специализированную коллекцию Parameters. На рисунке 3.1 показано как описать параметр команды и передать в него фактическое значение.

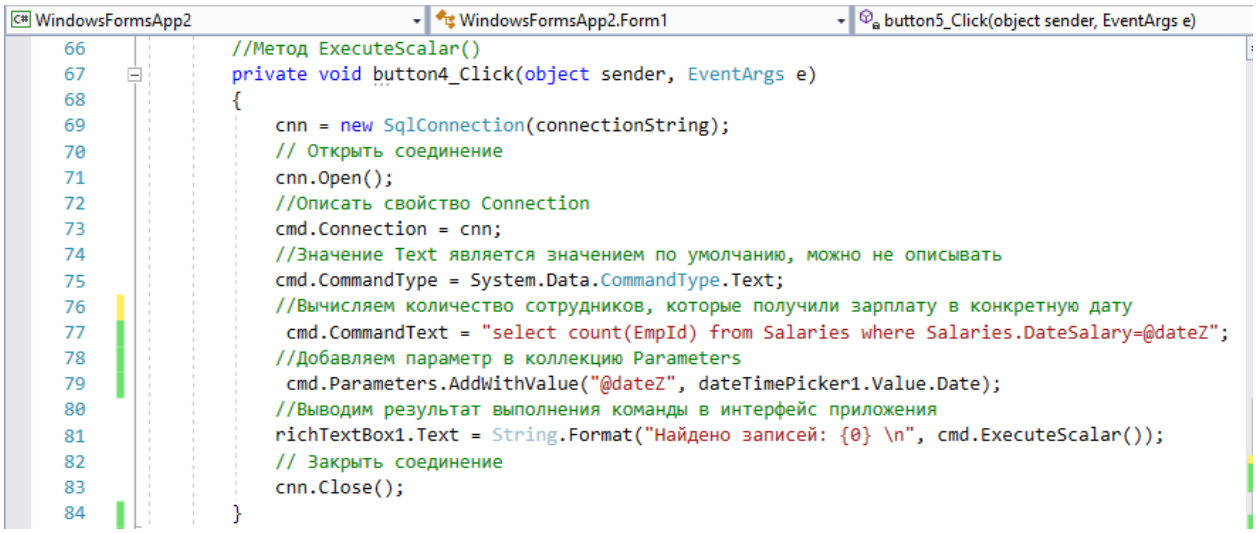


Рисунок 3.1 – Использование команды с параметром

Для ввода конкретного значения даты необходимо добавить в интерфейс приложения соответствующие элементы, например, dateTimePicker для win-формы или Calendar для web-формы, как показано на рисунке 3.2.

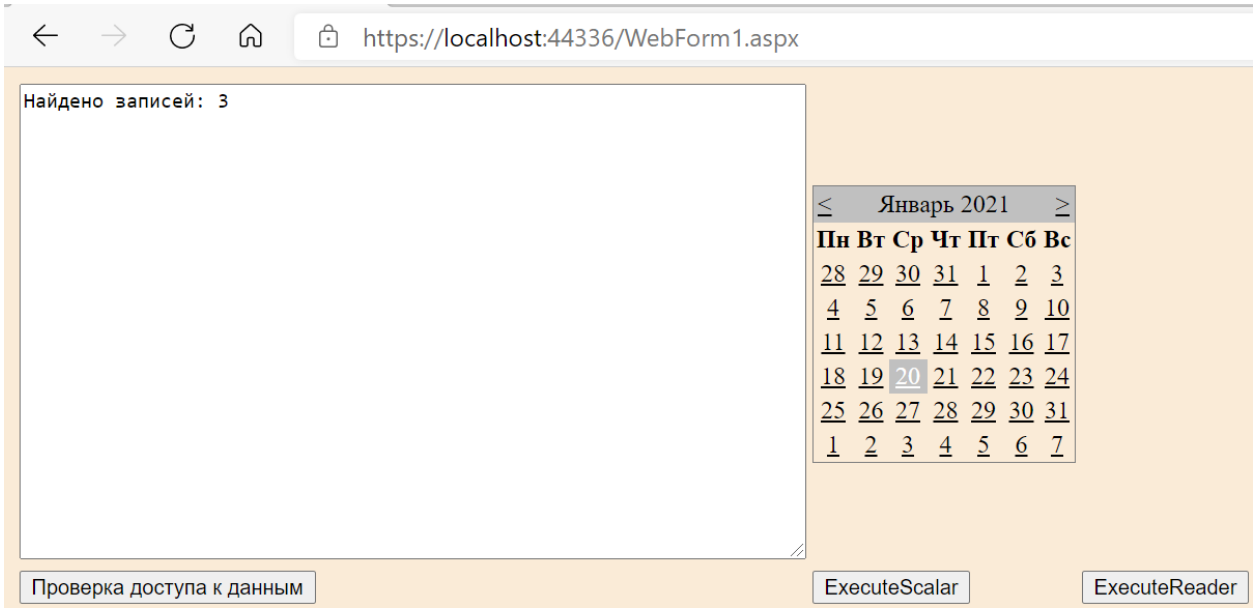


Рисунок 3.2 – Веб-интерфейс с добавленным элементом Calendar

3.2 Объект Command и вызов хранимых процедур

Хранимые процедуры являются хорошо инкапсулированными блоками логики, которые могут принимать данные (через входные параметры) и возвращать данные (через результирующие наборы и|или выходные параметры).

При описании свойств объекта Command используется значение CommandType.StoredProcedure, а имя хранимой процедуры присваивается свойству CommandText.

Для создания хранимой процедуры средствами Visual Studio .NET выберите на вкладке Обозреватель серверов папку Хранимые процедуры и добавьте новую процедуру. Можно использовать SQL Server Management Studio как показано на рисунке 3.3.

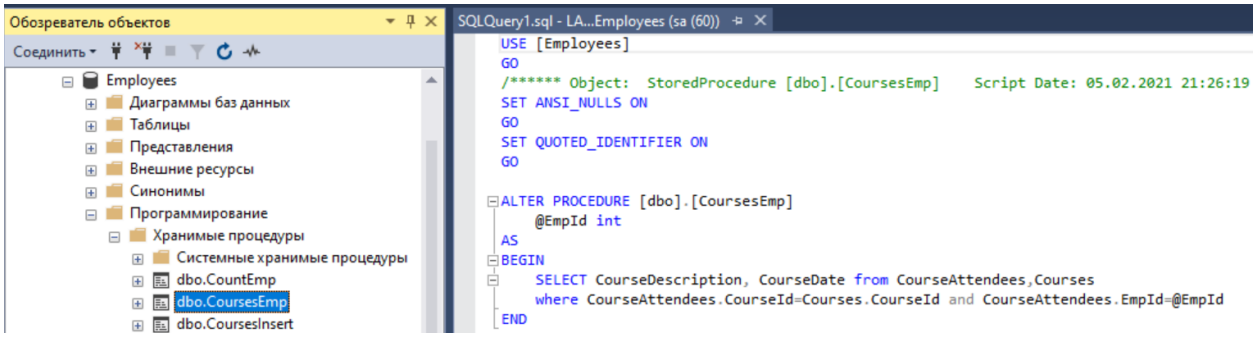


Рисунок 3.3 – Создание хранимой процедуры в менеджере сервера MS SQL Server

В любом случае среда разработки предложит шаблон для ввода текста процедуры. В шаблоне следует изменить имя процедуры, описать, если это необходимо, входные и выходные параметры, после оператора AS ввести код процедуры (рисунок 3.3).

Для вызова хранимой процедуры с параметрами необходимо (рисунок 3.4):

- описать свойства объекта Command: CommandType и CommandText,
- добавить имена параметров в коллекцию Parameters объекта Command,
- указать, как получить фактические значения параметров,
- запустить подходящий метод объекта Command.

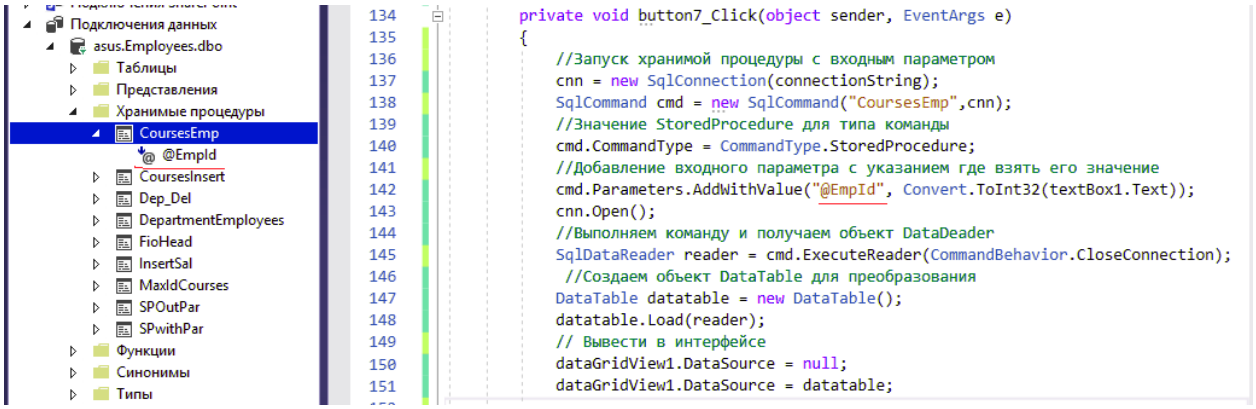


Рисунок 3.4 – Описание команды для вызова процедуры с входным параметром

В интерфейсе формы надо организовать ввод значения входного параметра и отображение результата выполнения хранимой процедуры, например, как на рисунке 3.5.

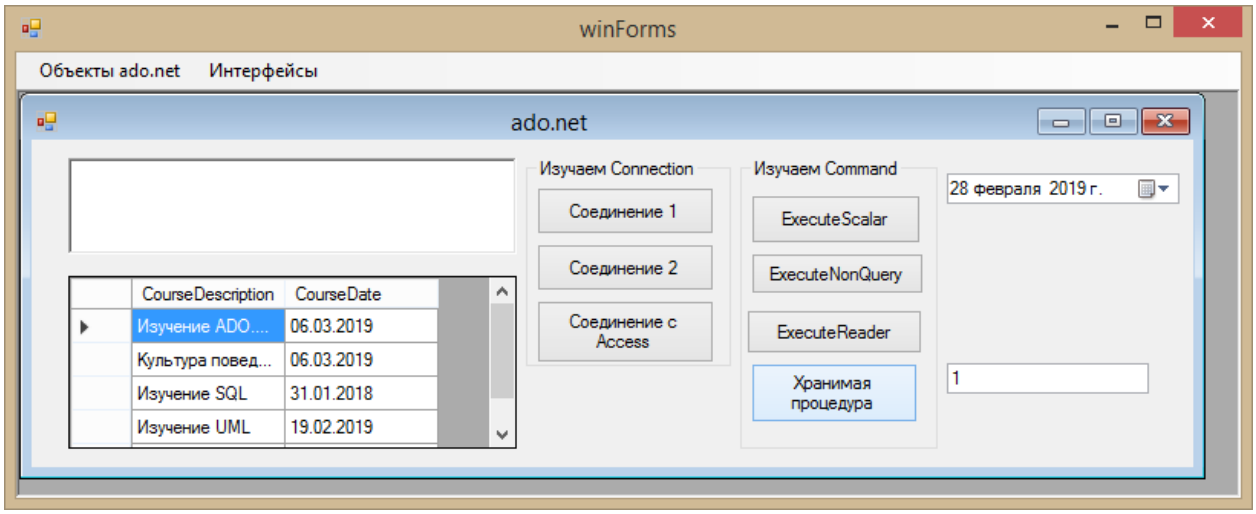


Рисунок 3.5 – Ввод значения входного параметра в win-форме

Особенности использования в хранимых процедурах выходных параметров можно увидеть на рисунках 3.6, 3.7 и 3.8. На рисунке 3.6 показан текст хранимой процедуры с входным и выходным параметрами: вычисляется количество сотрудников, которые прошли повышение квалификации в указанную дату. На рисунке 3.7 показан способ описания объекта Command для процедуры с выходным параметром. На рисунке 3.8 – интерфейс windows-формы при запуске процедуры.

```
CREATE PROCEDURE SPOutPar

    @DateZ date,
    @countFio int OUTPUT
AS
BEGIN
    SELECT @countFio=count(EmpId)
    FROM Courses, CourseAttendees
    WHERE CourseAttendees.CourseId=Courses.CourseId AND CourseDate=@DateZ
END
```

Рисунок 3.6 – Текст хранимой процедуры с входными и выходными параметрами

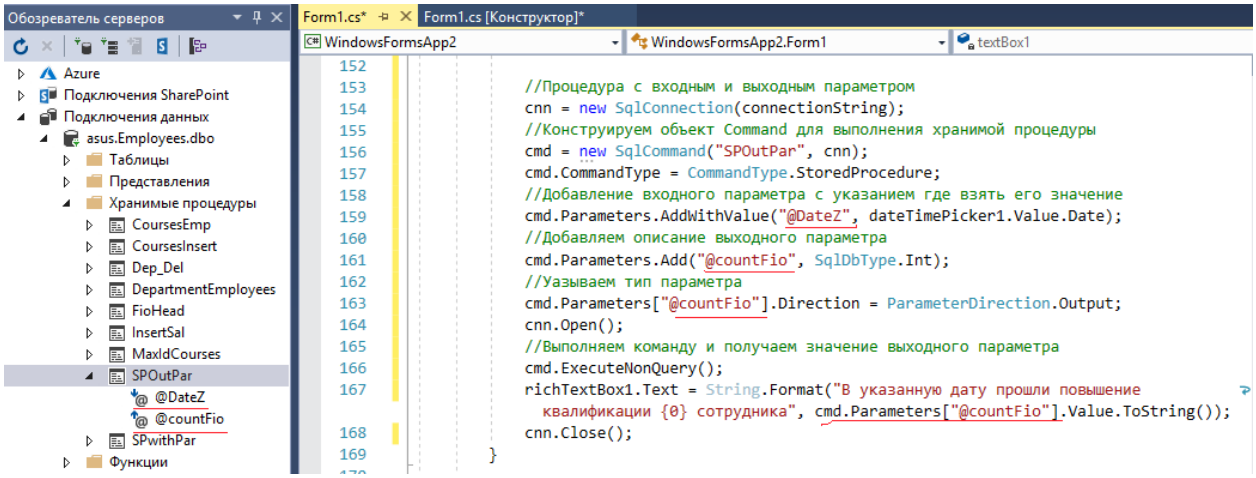


Рисунок 3.7 – Запуск хранимой процедуры с входными и выходными параметрами

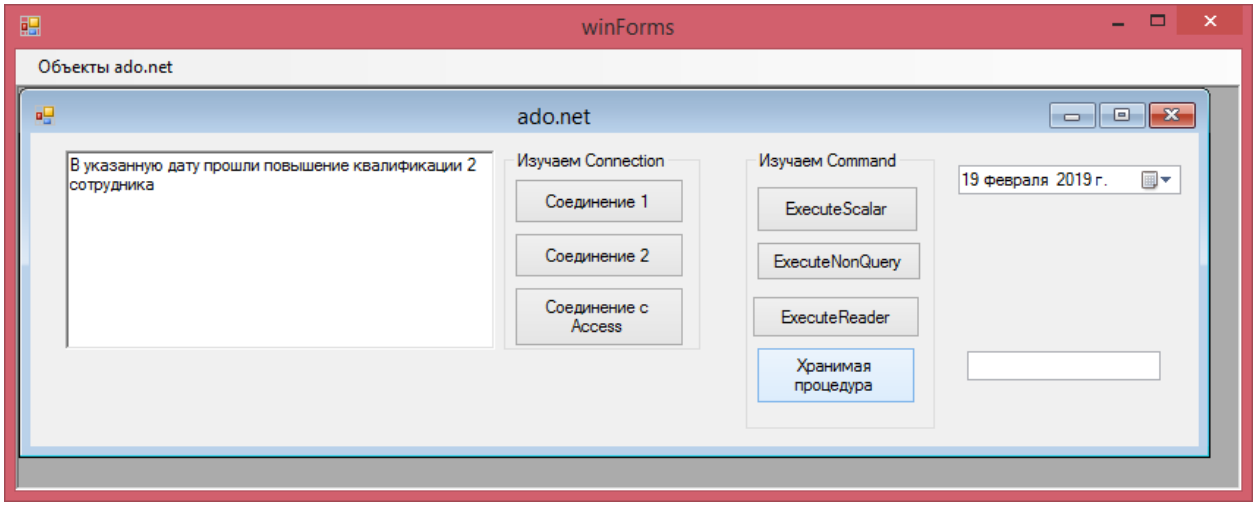


Рисунок 3.8 – Интерфейс формы при запуске процедуры с параметрами

3.3 Транзакции и ADO.NET. Локальные транзакции

Транзакция состоит из одной команды или группы команд, которые выполняются как пакет. Транзакции позволяют объединить несколько операций в одну единицу работы. Если в какой-либо точке транзакции

возникает ошибка, может быть, выполнен откат всех обновлений к их состоянию до начала транзакции.

В ADO.NET транзакции управляются объектом Connection, который для открытия транзакции имеет метод ***BeginTransaction().***

После начала транзакции при помощи свойства Transaction объекта Command к ней можно прикрепить команду. Затем, в зависимости от успеха или ошибки выполнения транзакции, можно зафиксировать или отказаться от изменений, сделанных в источнике данных.

Рассмотрим пример локальной транзакции. На рисунках 3.9 и 3.10 показан windows- интерфейс для запуска локальной транзакции. Команды транзакции (рисунок 3.11):

1. в таблице Courses выполняется поиск записей за указанный год;
2. создается новая таблица;
3. записи копируются в новую таблицу и удаляются из исходной.

```
ссылка: 1
private void Form7_Load(object sender, EventArgs e)
{
    cnn = new SqlConnection(str);
    listBox1.Items.AddRange(new object[]{ 2016,2017,2018});
    listBox1.SelectedIndex = 0;
}
```

Рисунок 3.9 – Инициирование данных в форме

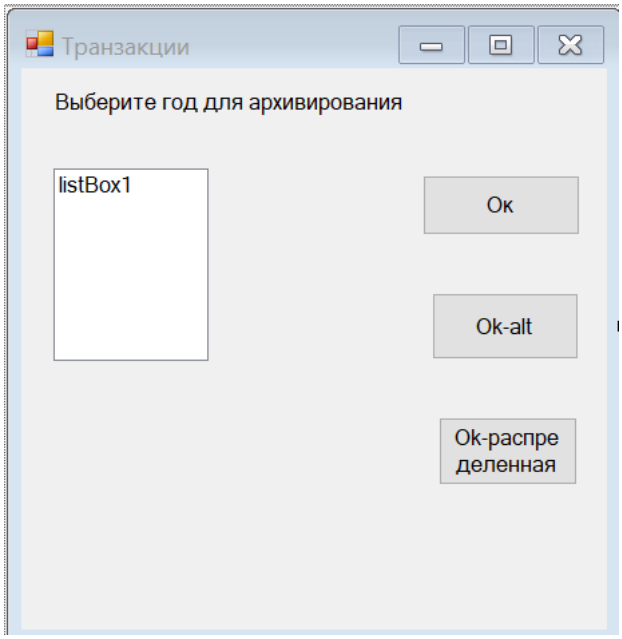


Рисунок 3.10 - Интерфейс для выбора года архивирования записей

```
private void button1_Click(object sender, EventArgs e)
{
    int records = 0;//Переменная для хранения количества записей, до начала транзакции=0
    // Пользователь выбирает год из списка
    string SelectedYear = listBox1.SelectedItem.ToString();
    cmd.Connection = cnn;
    cnn.Open();//'Открытие объекта Connection и запуск транзакции
    SqlTransaction trans = cnn.BeginTransaction();
    cmd.Transaction = trans;//Включение команды в транзакцию
    //Указание команды SQL для вставки соответствующих записей в архивную таблицу
    string sql = "SELECT * INTO Courses" + SelectedYear + " From Courses WHERE year(CourseDate)=" + SelectedYear;
    cmd.CommandText = sql; //Передача текста команды SQL в транзакцию
    int result = cmd.ExecuteNonQuery();
    if (result > 0)//Отображение результатов вставки записей в архивную таблицу
    {
        records = result;
        MessageBox.Show(records + " записей вставлено в таблицу успешно за " + SelectedYear);
    }
        else
        {
            MessageBox.Show("Записи не вставлены в таблицу  за " + SelectedYear);
            trans.Rollback();
        }//при отсутствии записей транзакцию следует прекратить
        if (records > 0)//Удаление записей из исходной таблицы
        {
            sql = "DELETE FROM Courses WHERE year(CourseDate)=" + SelectedYear;
            cmd.CommandText = sql; //Эта команда находится в той же транзакции
            result = cmd.ExecuteNonQuery();
            if (result == records)//Показать результаты удаления
            {
                MessageBox.Show(records + "записей удалено");
                trans.Commit();
            }//Все действия успешно выполнены. Фиксируем транзакцию
            else MessageBox.Show("Записи не удалены");
            cnn.Close();
        }
```

Рисунок 3.11 – Программирование локальной транзакции

4. АВТОНОМНАЯ МОДЕЛЬ ДОСТУПА ADO.NET И КЛАСС DATASET

Технология ADO.NET предоставляет в распоряжение разработчика класс ***DataSet***, который спроектирован как автономный (отключенный) контейнер данных. В нем отсутствует соединение с базой данных. Фактически данные, хранящиеся в DataSet, даже не обязательно должны быть взяты из базы данных, с тем же успехом они могут представлять записи из файла CSV (Comma-Separated Values — значения, разделённые запятыми — файл текстового формата, предназначенный для представления табличных данных), файла XML, либо быть данными, полученными от измерительных устройств.

4.1 Объекты DataSet и DataAdapter

Класс ***DataAdapter*** связывает компоненты ADO.NET, подключенные к источнику данных, и автономный компонент DataSet. Основным назначением объекта DataAdapter является управление процессом передачи данных от источника данных к объекту DataSet и от объекта DataSet к источнику данных.

Для изучения принципов программирования DataSet добавьте windows-форму и web-форму в ваши проекты и разместите в них элементы DataGridView и GridView соответственно. На рисунке 4.1 приведен код загрузки данных таблицы в DataGridView для windows-формы. На рисунке 4.2 демонстрируется win-форма в режиме пользователя.

```
SqlDataAdapter da;
DataSet ds = new DataSet();
SqlCommandBuilder bild;
ссылка: 1
private void Form3_Load(object sender, EventArgs e)
{
    cnn = new SqlConnection(ConfigurationManager.ConnectionStrings
      ["Employees"].ConnectionString);
    da = new SqlDataAdapter("select * from Courses", cnn);
    da.Fill(ds, "Courses");
    dataGridView1.DataSource = ds.Tables["Courses"];
    dataGridView1.Columns[0].Visible = false;
    bild = new SqlCommandBuilder(da);
}
//При закрытии формы обновление записей базы данных
ссылка: 1
private void Form3_FormClosing(object sender, FormClosingEventArgs e)
{
    da.Update(ds.Tables["Courses"]);
}
```

Рисунок 4.1 – Загрузка данных в объект DataSet в windows-форме

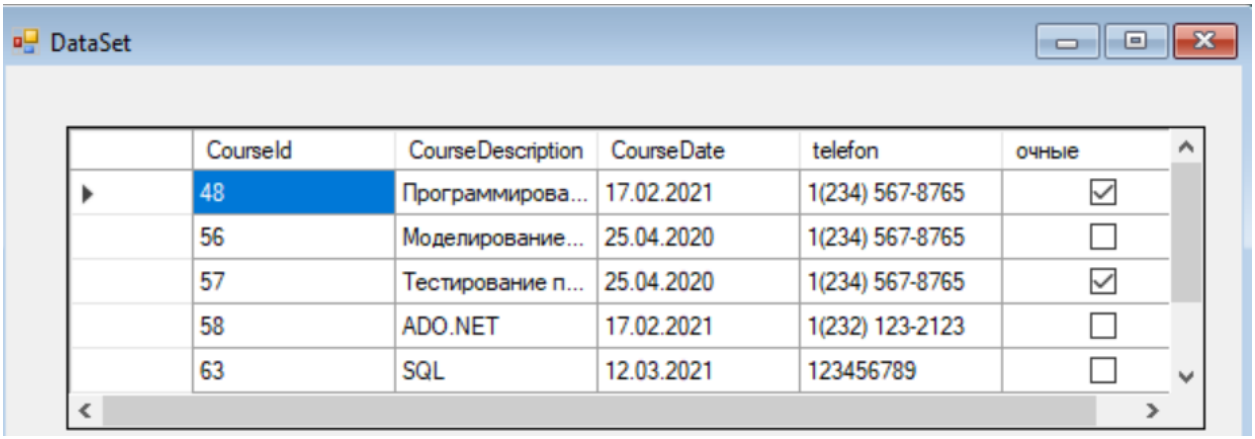


Рисунок 4.2 – Отображение DataSet в DataGridView windows-формы

Загрузить данные для локального хранения в объекте DataSet можно с помощью метода Fill() адаптера данных. Сохранить все изменения в БД можно с помощью метода Update() адаптера данных. Организовать отображение полученных данных удобно в элементе DataGridView, передать измененные данные в таблицу базы данных можно с помощью объектов CommandBuilder и DataAdapter.

Для web-формы программный код загрузки приведен на рисунке 4.3, а внешний вид приложения показан на рисунке 4.4.

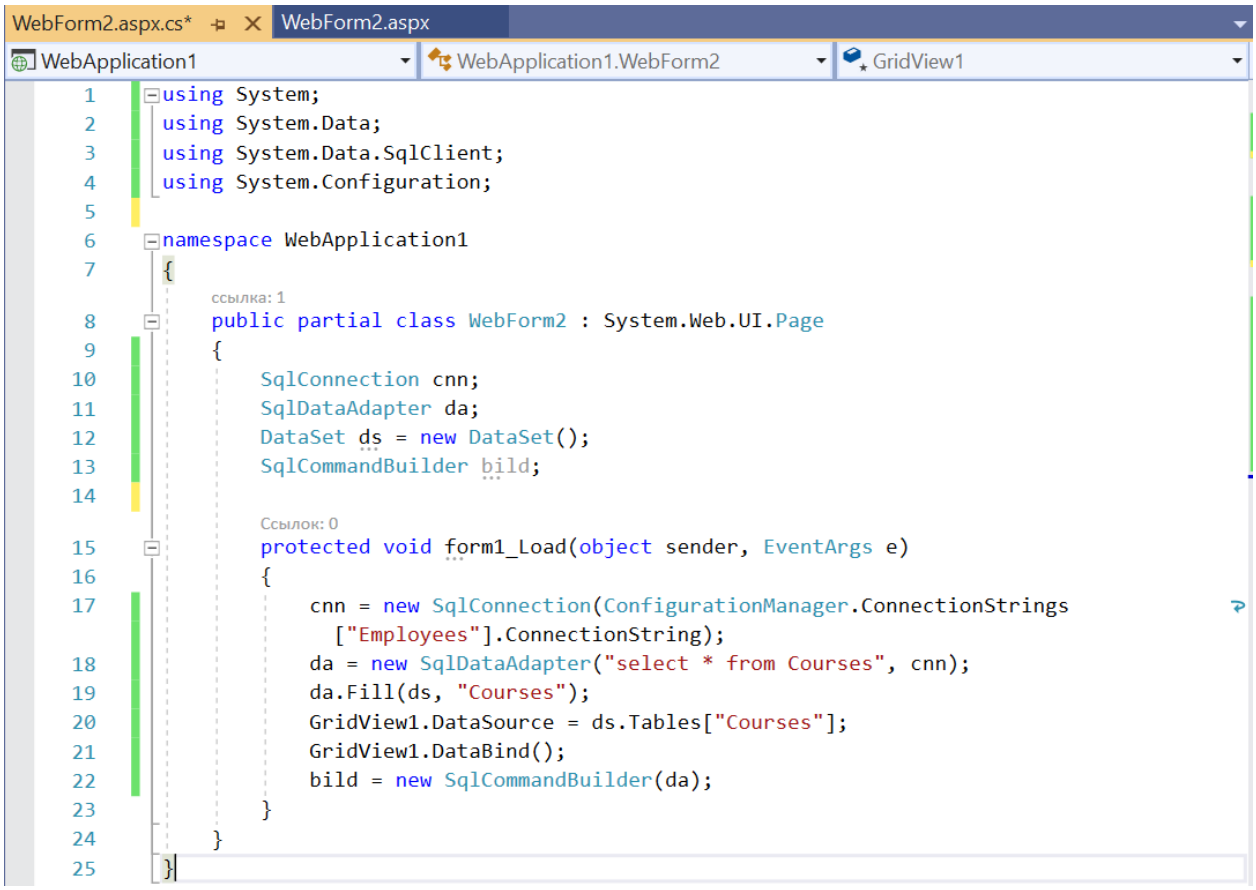
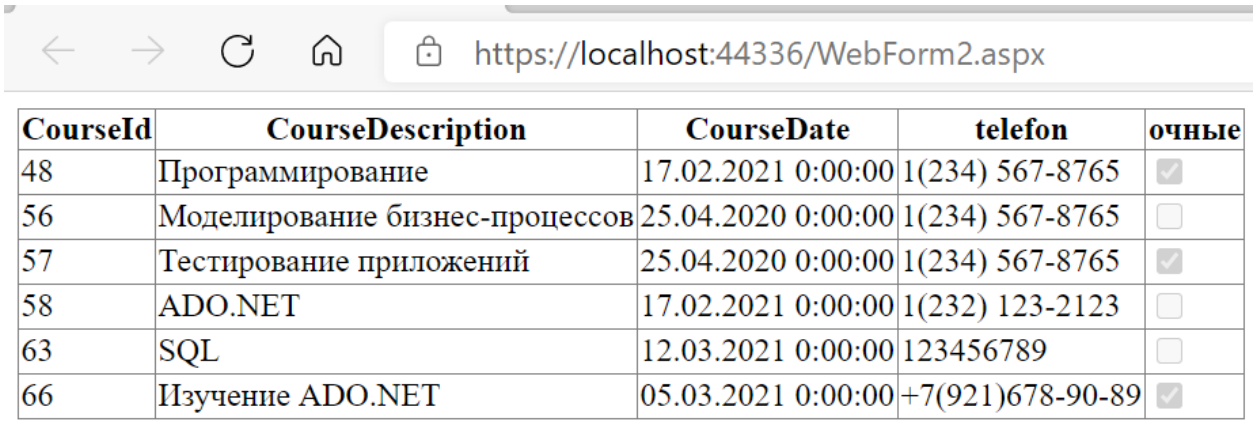


Рисунок 4.3 – Web-форма для работы с отсоединенными данными



CourseId	CourseDescription	CourseDate	telefon	очные
48	Программирование	17.02.2021 0:00:00	1(234) 567-8765	☑
56	Моделирование бизнес-процессов	25.04.2020 0:00:00	1(234) 567-8765	☐
57	Тестирование приложений	25.04.2020 0:00:00	1(234) 567-8765	☑
58	ADO.NET	17.02.2021 0:00:00	1(232) 123-2123	☐
63	SQL	12.03.2021 0:00:00	123456789	☐
66	Изучение ADO.NET	05.03.2021 0:00:00	+7(921)678-90-89	☑

Рисунок 4.4 - Внешний вид web-приложения для исследования DataSet

4.2 Программирование вставки данных в объекте DataTable

Данные в объекте DataSet организованы в одной или нескольких объектах DataTable. Каждая Запись в DataTable представлена объектом DataRow. Каждый объект DataRow представляет одиночную запись в таблице, которая была извлечена из источника данных. DataRow является контейнером для действительных значений полей.

Рассмотрим возможности объекта DataTable для работы со строками: начнем с добавления и удаления строк в объекте. На рисунке 4.5 показана windows-форма с элементами для ввода значений в поля таблицы, отображаемой в DataGridView.

На рисунке 4.6 приведен программный код формирования значений и добавление новой строки в DataTable. Технологию записи новой строки данных в объект DataTable можно описать следующими шагами:

- Создается новый экземпляр объекта DataRow методом NewRow().
- Присваиваются конкретные значения полям новой записи из элементов интерфейса.
- Запись включается в коллекцию Rows таблицы с помощью метода Add().

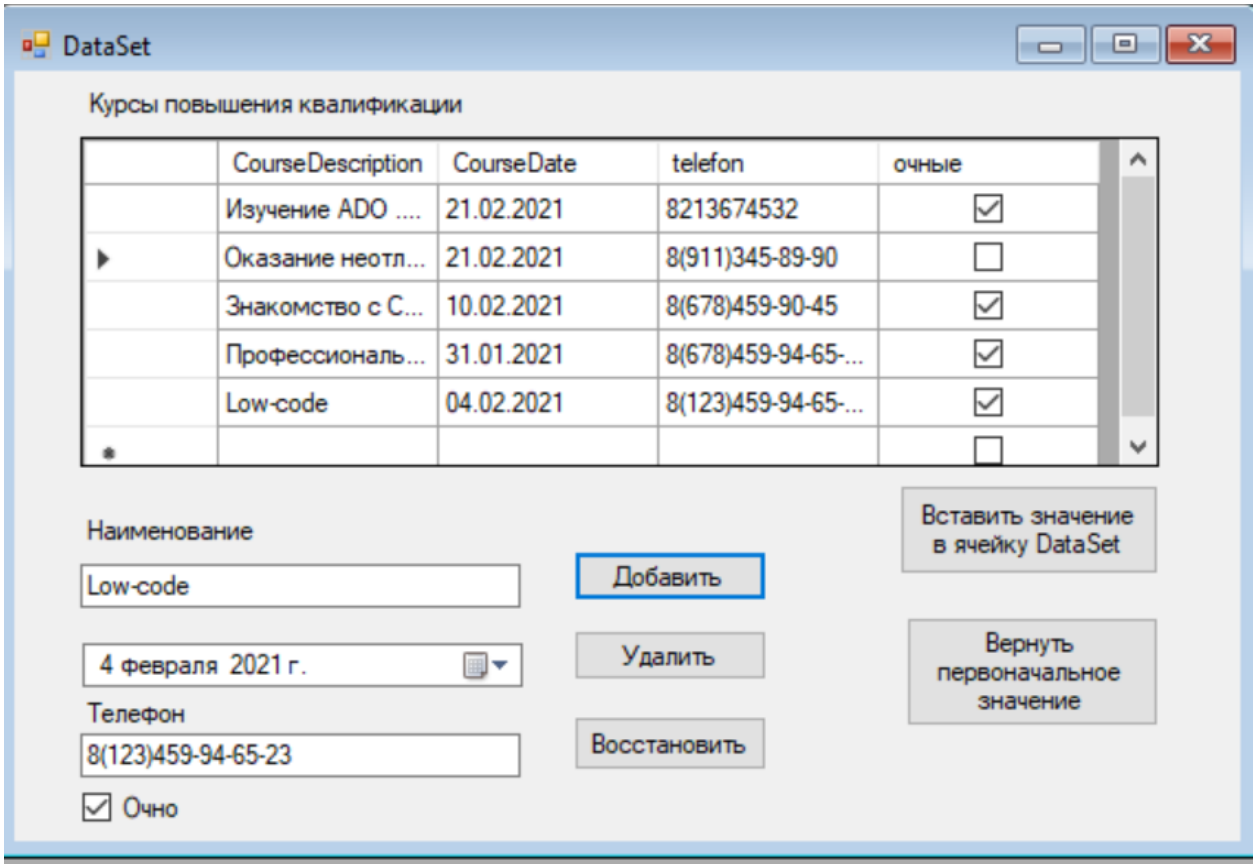


Рисунок 4.5 – Приложение для исследования объектов DataTable и DataSet

```
//Добавить новую запись
ссылка: 1
private void button1_Click(object sender, EventArgs e)
{
    //Получить новую строку
    DataRow dr = ds.Tables["Courses"].NewRow();
    //Получить значения полей
    dr[1] = textBox2.Text;
    dr[2] = dateTimePicker1.Value.Date;
    dr[3] = textBox3.Text;
    dr[4] = checkBox1.Checked;
    //Добавить новую строку в коллекцию строк DataSet
    ds.Tables["Courses"].Rows.Add(dr);
}
```

Рисунок 4.6 – Добавление новой строки в DataTable в win-форме

Класс ***Page*** используется в модели веб-страницы для программирования доступа к серверным элементам страницы. Метод Redirect(url) на рисунке 4.7 перенаправляет запрос новому URL-адресу. Свойство Page.Request возвращает объект HttpRequest для запрашиваемой страницы. Обновление текущей страницы выполняется методом Update() адаптера данных.

```
//Добавление новой строки
Ссылок: 0
protected void Button1_Click(object sender, EventArgs e)
{
    //Получить новую строку
    DataRow dr = ds.Tables["Courses"].NewRow();
    //Получить значения полей
    dr[1] = TextBox1.Text;
    dr[2] = Calendar1.SelectedDate;
    dr[3] = TextBox2.Text;
    dr[4] = CheckBox1.Checked;
    //Добавить новую строку в коллекцию строк DataSet
    ds.Tables["Courses"].Rows.Add(dr);
    GridView1.DataBind();
    da.Update(ds.Tables[0]);
}
//Обновление состояния текущей страницы
Ссылок: 0
protected void Button2_Click(object sender, EventArgs e)
{
    Page.Response.Redirect(Page.Request.Url.ToString());
}
```

Рисунок 4.7 – Программирование веб-формы для добавления строки и обновления страницы

Строки можно не только вставлять, но и удалять целиком. Удаление записи можно выполнить методом Delete() объекта DataRow. Этот метод отмечает указанную строку на удаление, которое на самом деле произойдет только после вызова метода AcceptChanges().

Метод AcceptChanges() фиксируют изменения – рисунок 4.8, а метод RejectChanges() наоборот, отменяет последние изменения в классах DataTable, DataSet и DataRow.

```
//Пометка указанной строки на удаление
ссылка: 1
private void button2_Click(object sender, EventArgs e)
{
    //Вызов метода AcceptChange() для фиксирования всех изменений
    ds.Tables["Courses"].AcceptChanges();
    //Пометка на удаление строки, указанной щелчком мыши в dataGridView
    ds.Tables["Courses"].Rows[dataGridView1.CurrentCell.RowIndex].Delete();
}
//Снять пометку удаления и восстановить строку
ссылка: 1
private void button3_Click(object sender, EventArgs e)
{
    ds.Tables[0].RejectChanges();
}
```

Рисунок 4.8 – Удаление и восстановление строки

4.3 Синтаксис адресации полей DataSet и элементов DataGridView

При построении интерфейса могут возникать различные задачи, связанные с записью данных в конкретное поле объекта DataSet или конкретную ячейку DataGridView. Рассмотрим принципы адресации к этим элементам.

Данные в DataTable представлены как «непрямоугольный» массив, а следовательно, обращение к отдельным ячейкам выглядит как на рисунке 4.9:

```
//Вставить конкретное значение в указанную ячейку DataSet
private void button4_Click(object sender, EventArgs e)
{
    ds.Tables[0].Rows[1]["telefon"] = textBox3.Text;
    dataGridView1.DataSource = ds.Tables[0];
}
```

Рисунок 4.9 – Адресация ячеек DataSet

Одним из привлекательных аспектов объекта DataRow является поддержка версий, которая позволяет получить различные значения заданного столбца определенной строки. Версии данных в DataRow собраны в системном перечислении DataRowVersion (рисунок 4.10).

```
//Вернуть первоначальное значение ячейки в DataSet
private void button5_Click(object sender, EventArgs e)
{
    ds.Tables[0].Rows[1]["telefon"] = ds.Tables[0].Rows[1]["telefon",
      DataRowVersion.Original];
    dataGridView1.DataSource = ds.Tables[0];
}
```

Рисунок 4.10 – Системное перечисление DataRowVersion

При необходимости обращения к конкретному полю DataGridView можно использовать свойства текущей ячейки CurrentCellAddress.X и CurrentCellAddress.Y как на рисунке 4.11.

```
//Вставить конкретное значение в указанную ячейку DataSet
private void button4_Click(object sender, EventArgs e)
{
    //ds.Tables[0].Rows[1]["telefon"] = textBox3.Text;
    //dataGridView1.DataSource = ds.Tables[0];

    //Адрес активной ячейки в DataSet
    dataGridView1[dataGridView1.CurrentCellAddress.X,
      dataGridView1.CurrentCellAddress.Y].Value = textBox3.Text;
}
```

Рисунок 4.11 – Вставка значений в ячейки DataSet

5. АВТОНОМНАЯ МОДЕЛЬ ДОСТУПА К РЕЛЯЦИОННЫМ ДАННЫМ

В типовом решении данные запрашиваются приложением клиента посредством URL. Данные, полученные по запросу, попадают в соответствующий компонент промежуточного слоя, который конвертирует XML и с помощью адаптера размещает данные в DataSet.

В клиентском приложении данные отображаются с использованием комбинации элементов управления, например, DataGrid, DataGridView. Пользователь может добавлять, удалять, или редактировать данные. После окончания работы с данными пользователем, DataSet снова конвертируется в документ XML для передачи обратно к компоненту сервера.

Компонент промежуточного слоя конвертирует документ XML обратно в DataSet, и использует адаптер, чтобы попытаться применить изменения данных в БД. При обнаружении коллизий, компонент сервера пытается урегулировать их, используя встроенные бизнес-правила. Урегулированный DataSet будет возвращён обратно клиенту. После этого пользователь может продолжить работу с DataSet, причём, DataSet после этого будет отражать самую последнюю версию данных в БД.

5.1 Извлечение данных из форм, являющихся справочниками

Для извлечения связанных записей из формы-справочника можно предложить следующую технологию. На рисунке 5.1 представлена форма Руководители отделов, через которую пользователь вносит данные в таблицу Отделы (Dep).

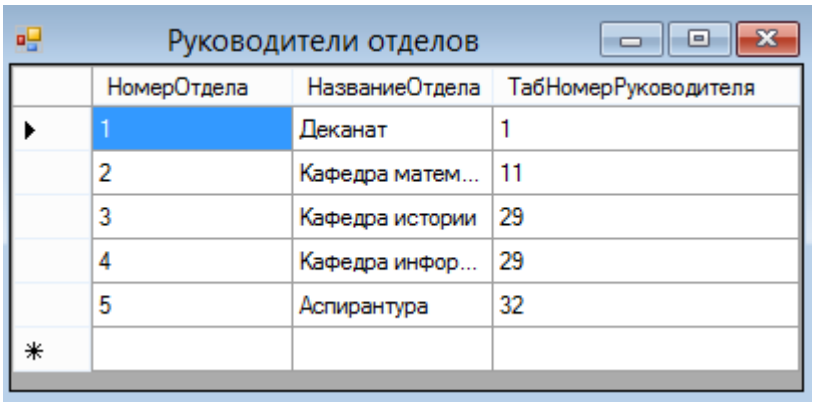


Рисунок 5.1 - Интерфейс для заполнения таблицы Dep

Для выбора данных о руководителе отдела и записи его табельного номера разработана форма-справочник Сотрудники (рисунок 5.2). Программный код формы-справочника представлен на рисунке 5.3.

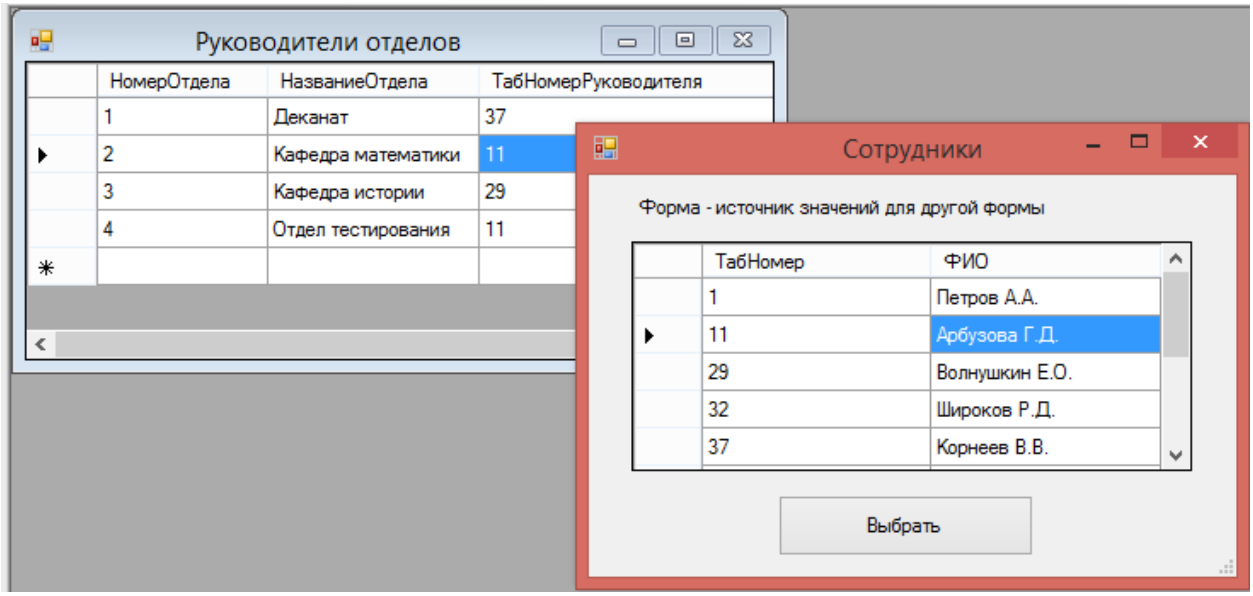


Рисунок 5.2 - Форма-справочник для таблицы Сотрудники (Emp)

```
//Описание свойства формы для хранения выбранного пользователем значения из
  DataGridView
//Свойство только для чтения (read-only): есть блок get, но нет блока set
private string sEmp = "";
public string SelectedEmp
{
    get { return this.sEmp; }
}
private void Form5_Load(object sender, EventArgs e)
{
    cnn = new SqlConnection(ConfigurationManager.ConnectionStrings
      ["Employees"].ConnectionString);
    da = new SqlDataAdapter("select EmpId ТабНомер, FIO ФИО from Emp", cnn);
    ds.Clear();
    da.Fill(ds, "Emp");
    dataGridView1.DataSource = ds.Tables["Emp"];
    //Сбросить предыдущий выбор значения свойства
    this.sEmp = "";
    //Назначение кнопке свойства DialogResult позволяет при нажатии кнопки
      закрывать форму-справочник
    Выбрать.DialogResult = System.Windows.Forms.DialogResult.OK;
}

private void Выбрать_Click(object sender, EventArgs e)
{
    //Запись выбранного пользователем значения из DataGridView в свойство
      SelectedEmp
    this.sEmp = ds.Tables["Emp"].Rows[this.BindingContext[ds.Tables
      [0]].Position]["ТабНомер"].ToString();
}
```

Рисунок 5.3 – Программный код формы-справочника

В форме Руководители отделов при щелчке мышью по соответствующей ячейке в столбце ТабНомерРуководителя должна открываться форма-справочник Сотрудники (рисунок 5.4):

```
//При щелчке по соответствующей ячейке экземпляр формы-справочника активизируется в модальном режиме
private void dataGridView1_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (dataGridView1.CurrentCell.ColumnIndex == dataGridView1.Columns["DepartmentHeadId"].DisplayIndex)
   {
        f.ShowDialog();
        if (f.DialogResult == DialogResult.OK)
        {
dataGridView1[dataGridView1.CurrentCellAddress.X, dataGridView1.CurrentCellAddress.Y].Value = f.SelectedEmp;
         }
    }
}
```

Рисунок 5.4 – Вызов формы-справочника

В рассмотренном примере обратите внимание на адресацию к ячейкам DataGridView (выровненный массив) и адресацию к данным DataSet (не выровненный массив).

5.2 Привязка данных. Простая привязка данных в формах Windows

Привязка данных дает возможность отображать и изменять информацию из источника данных в интерфейсе пользователя. Можно выполнить привязку данных не только для таблиц баз данных, но и практически для любых других структур данных.

В проектах, построенных на основе Windows-форм, используют простую (simple data binding) и составную (complex data binding) привязки данных. Простой привязкой данных (simple data binding) называется связывание элемента управления с единственным элементом данных.

Для реализации привязки данных удобно использовать класс BindingSource, который обеспечивает не только связь между элементом управления и данными, но и навигацию по коллекции объектов с возможностью добавления или удаления объектов. Навигацию по коллекции реализуют методы: MoveNext(), MoveLast(), MovePrevious() и MoveFirst(), с помощью которых можно отслеживать значения свойств Position и Current.

У каждого элемента управления (наследника класса Control) есть свойство DataBindings – коллекция экземпляров класса Binding, привязок к данным. При описании привязки можно указать источник данных (data source), свойство (поле) источника данных и привязываемое свойство элемента управления, например, надпись на кнопке.

На рисунке 5.3 показан пример формы с простой привязкой данных таблицы к элементам формы: текстовому полю TextBox, календарю DateTimePicker, текстовому полю с маской MaskedTextBox, флажку ChekBox, списку ListBox.

Реализацию привязок программным способом можно увидеть на рисунке 5.6. Навигацию по записям bindingSource, добавление и удаление записей можно реализовать методами bindingSource или с помощью элемента bindingNavigator как на рисунке 5.5. Программный код рисунков 5.7-5.9 демонстрирует технологию перемещения по связанным массивам данных.

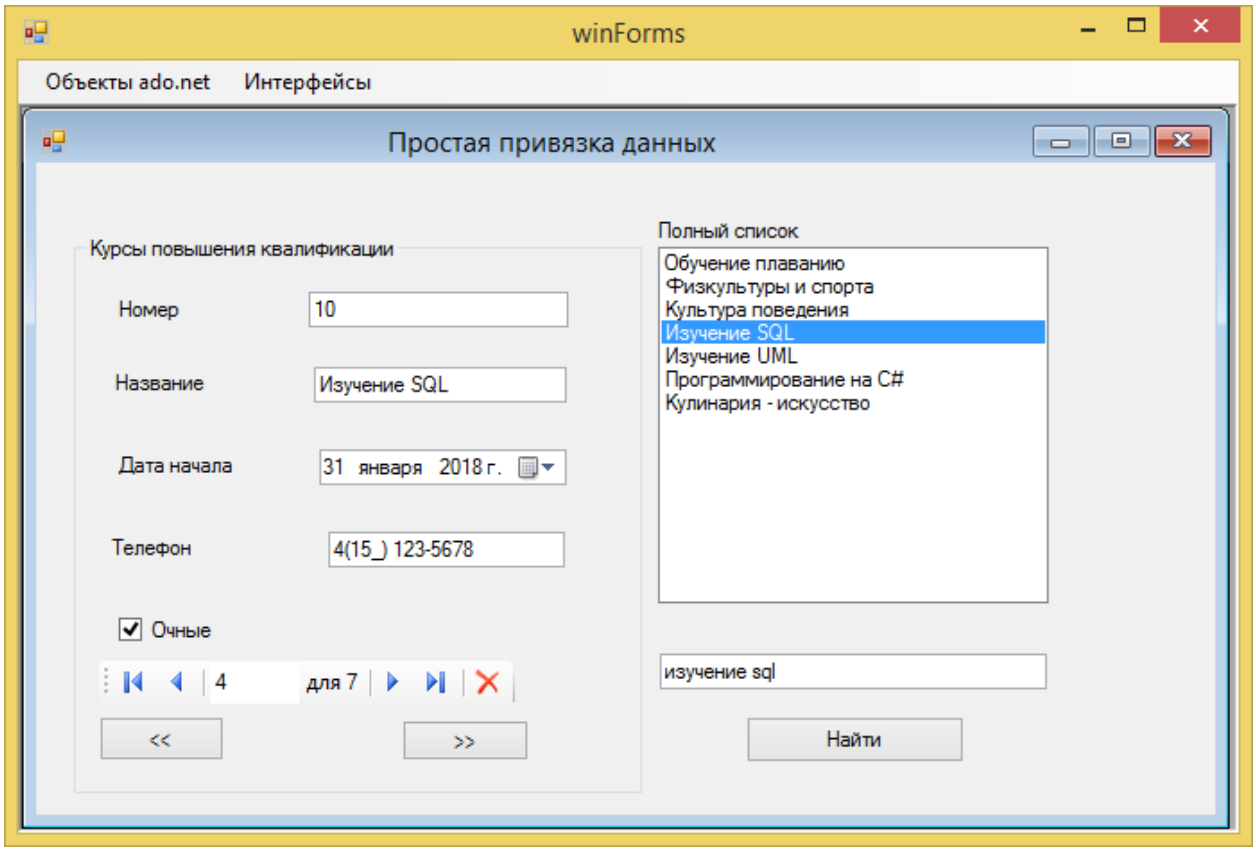


Рисунок 5.5 - Простая привязка данных к элементам формы



Рисунок 5.6 – Соответствие полей таблицы элементам интерфейса

```
//Щелчок по элементу списка отображает полную запись
private void listBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    bindingSource1.Position = listBox1.SelectedIndex;
}
```

Рисунок 5.7 – Выбор элемента списка

```
//Найти запись
private void button3_Click(object sender, EventArgs e)
{
    bindingSource1.Position = bindingSource1.Find("CourseDescription",
      textBox3.Text);
}
```

Рисунок 5.8 – Поиск выбранного элемента

```
//Перемещение по записям
private void button1_Click(object sender, EventArgs e)
{
    bindingSource1.MovePrevious();
}

private void button2_Click(object sender, EventArgs e)
{
    bindingSource1.MoveNext();
}
```

Рисунок 5.9 – Перемещение по записям

5.3 Привязка данных к элементу DataGridView в Windows Forms

Привязку данных к элементам формы в большинстве случаев выполняют с помощью компонента BindingSource, который управляет деталями взаимодействия с источником данных. Компонент BindingSource может представлять любой источник данных и обеспечивает большую гибкость при выборе или изменении расположения данных.

На рисунке 5.10 пример интерфейса, построенного на связанных и синхронизированных DataGridView. Синхронизация записей двух таблиц реализована с помощью BindingSource. На рисунке 5.11 приведен программный код привязки и синхронизации элементов управления DataGridView.

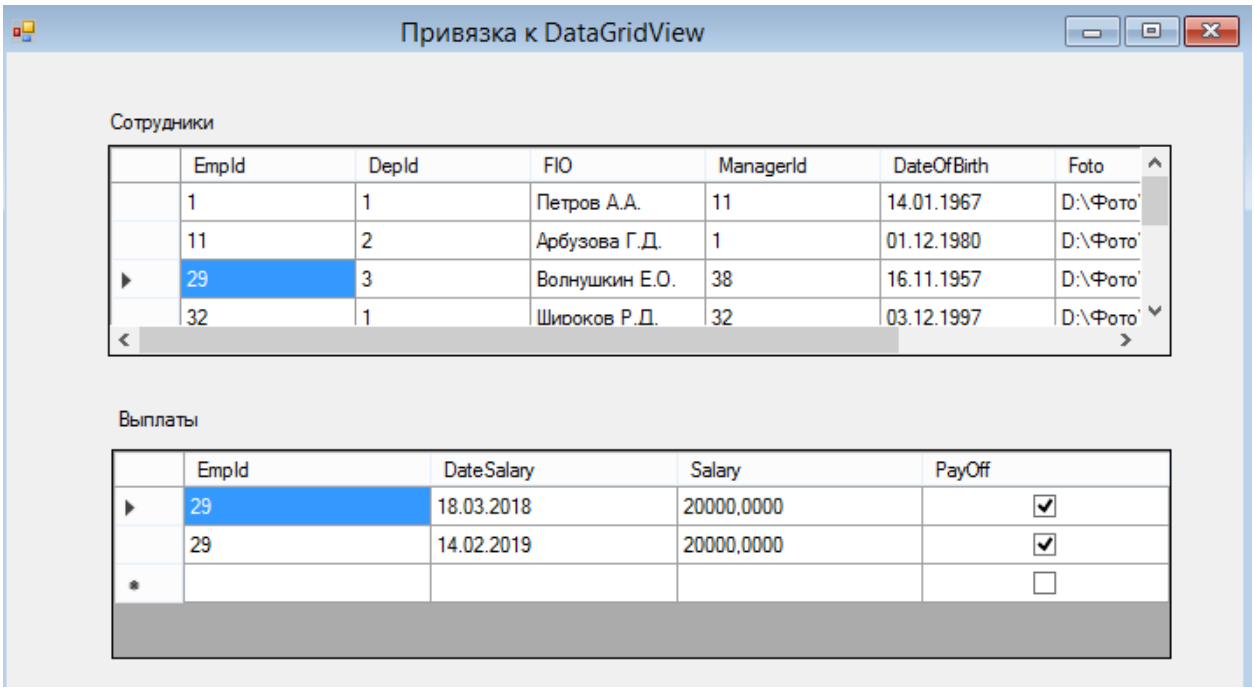


Рисунок 5.10 - Данные из двух связанных таблиц базы данных

```
    private void Form7_Load(object sender, EventArgs e)
    {
cnn = new SqlConnection(ConfigurationManager.ConnectionStrings["Employees"].ConnectionString);
        //Адаптеры данных
        daEmp = new SqlDataAdapter("select * from Emp", cnn);
        daSal = new SqlDataAdapter("select * from Salaries", cnn);
        daEmp.Fill(ds, "Emp");
        daSal.Fill(ds, "Sal");
        //Отношение между наборами данных в DataSet
ds.Relations.Add("E_S", ds.Tables["Emp"].Columns["EmpId"], ds.Tables["Sal"].Columns["EmpId"]);
        //Обеспечим возможность внесения изменений в таблицу Salaries
        bild = new SqlCommandBuilder(daSal);
        //Привязки элементов к данным DataGridView
        bind1.DataSource = ds.Tables["Emp"];
        dataGridView1.DataSource = bind1;
        bind2.DataSource = bind1;
        bind2.DataMember = "E_S";
        dataGridView2.DataSource = bind2;
    }
```

Рисунок 5.11 – Привязка элементов управления к данным

5.4 Привязка данных к элементу управления DetailsView в Web Form

Элемент DetailsView спроектирован для отображения отдельных записей в web-интерфейсе. Он размещает каждую часть информации в отдельной строке таблицы.

Элемент DetailsView может быть привязан к коллекции элементов. DetailsView позволяет перемещаться от одной записи к другой, используя страничные элементы управления при установке свойства AllowPaging в true.

Элементы управления страницами при этом можно конфигурировать, используя соответствующие свойства.

На рисунке 5.13 показан элемент DetailsView, привязанный к набору записей о курсах повышения квалификации. На рисунке 5.12 приведен программный код привязки и перехода по страницам элемента DetailsView.

```
protected void Page_Load(object sender, EventArgs e)
{
    SqlConnection cnn = new SqlConnection
      (WebConfigurationManager.ConnectionStrings
      ["Employees"].ConnectionString);
    SqlDataAdapter  da=new SqlDataAdapter("select * from Courses",cnn);
    DataSet ds = new DataSet();
    da.Fill(ds, "Courses");
    DetailsView1.DataSource = ds.Tables[0];
    DetailsView1.DataBind();
}
//Переход по страницам DetailsView
protected void DetailsView1_PageIndexChanging(object sender,
  DetailsViewPageEventArgs e)
{
    DetailsView1.PageIndex = e.NewPageIndex;
    DetailsView1.DataBind();
}
```

Рисунок 5.12 – Программирование web-страницы

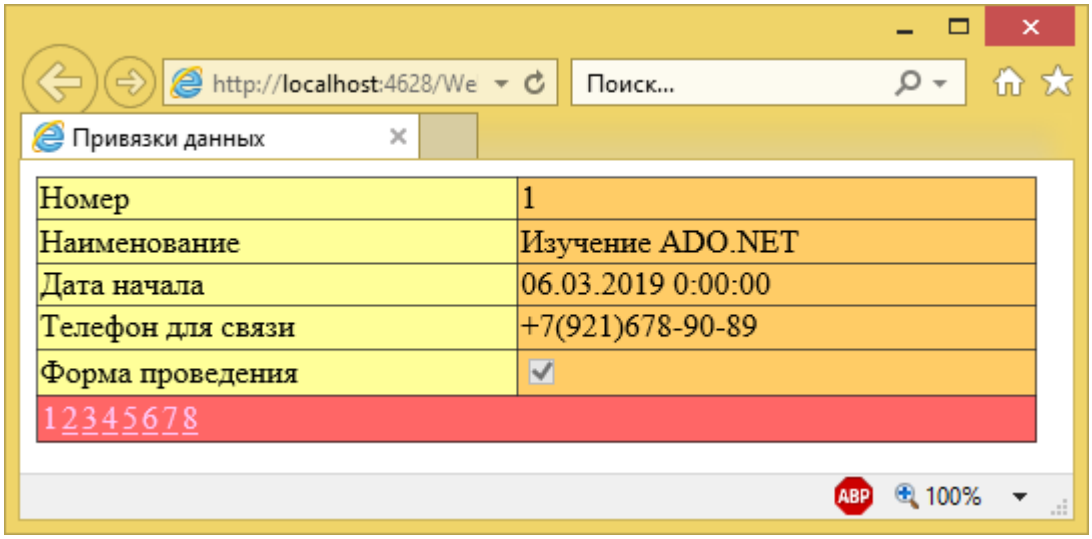


Рисунок 5.13 - Элемент DetailsView, привязанный к набору записей

5.5 Составная привязка данных в формах Windows

Составная привязка данных (complex data binding) используется при связывании элемента управления с несколькими элементами данных,

например, связывание элемента управления «комбинированный список» (comboBox) с полем, отображаемым для пользователя и с полем, значение которого будет использовано при операции обновления записи в самой базе данных.

В форме на рисунке 5.14 организована привязка данных и синхронизация для DataGridView и элемента управления comboBox. Элементы bindingSource можно разместить на форме при ее проектировании.

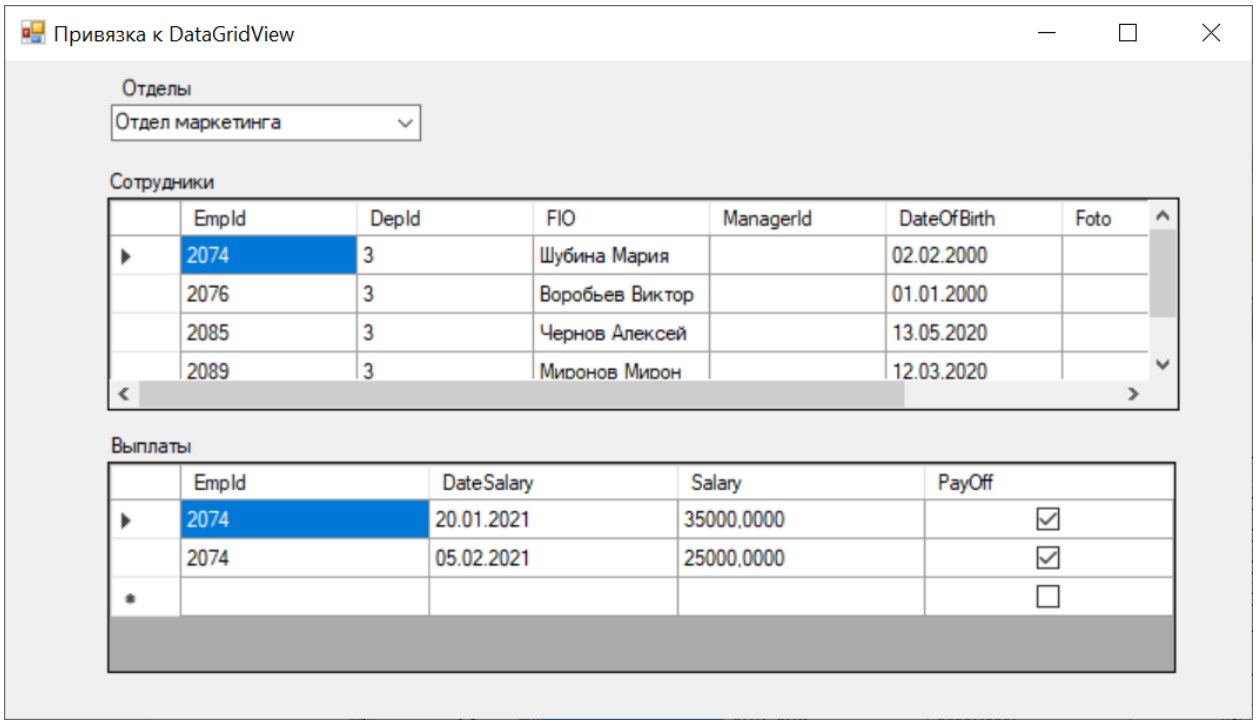


Рисунок 5.14 - Составная (complex) привязка данных к элементу формы comboBox

На рисунке 5.15 приведен программный код формы с простой и комплексной привязками элементов формы к элементам данных в DataSet.

```
    private void Form7_Load(object sender, EventArgs e)
    {
cnn = new SqlConnection(ConfigurationManager.ConnectionStrings
 ["Employees"].ConnectionString);
        //Адаптеры данных
        daDep = new SqlDataAdapter("select DepId, Description from Dep",cnn);
        daEmp = new SqlDataAdapter("select * from Emp", cnn);
        daSal = new SqlDataAdapter("select * from Salaries", cnn);
        daDep.Fill(ds, "Dep");
        daEmp.Fill(ds, "Emp");
        daSal.Fill(ds, "Sal");
        //Отношение между наборами данных в DataSet
        ds.Relations.Add("D_E",ds.Tables["Dep"].Columns["DepId"],ds.Tables["Emp"].Columns
          ["DepId"]);
ds.Relations.Add("E_S", ds.Tables["Emp"].Columns["EmpId"], ds.Tables["Sal"].Columns
 ["EmpId"]);
        //Обеспечим возможность внесения изменений в таблицу Salaries
        bild = new SqlCommandBuilder(daSal);
        //Привязки элементов к данным DataGridView
        bind1.DataSource = ds.Tables["Dep"];
        //Составная привязка
        comboBox1.DataSource = bind1;
        comboBox1.DisplayMember = "Description";
        comboBox1.ValueMember = "DepId";
        //Простая привязка
        bind2.DataSource = bind1;
        bind2.DataMember = "D_E";
        dataGridView1.DataSource = bind2;
        bind3.DataSource = bind2;
        bind3.DataMember = "E_S";
        dataGridView2.DataSource = bind3;
    }
```

Рисунок 5.15 – Простая и комплексная привязка элементов к данным

6. ПРЕДСТАВЛЕНИЕ ПОДМНОЖЕСТВА ДАННЫХ В ИНТЕРФЕЙСЕ ПОЛЬЗОВАТЕЛЯ

Разработка интерфейса приложений требует разнообразных средств для сортировки, фильтрации, поиска, изменения и навигации. Класс ***DataView*** реализует отбор табличных данных из объектов DataTable по заданным критериям с возможностью привязки отобранных данных к элементам интерфейса. Класс DataView являются настраиваемым представлением DataTable. DataView не сохраняет данные, а представляет связанное представление соответствующего DataTable. Изменения данных DataView влияют на содержание DataTable. Изменения данных DataTable влияют на все связанные с ним DataView.

Объект DataView обладает свойствами, которые позволяют настраивать способ отображения данных из объекта DataTable следующим образом:

- Изменять порядок сортировки (нисходящий или восходящий) по одному или нескольким полям.
- Использовать выражение для фильтрации записей, которое указывает критерии отображения записей на основе значений полей.
- Применять фильтр состояния записей, который указывает критерии отображения записей на основе состояния записи.

Объект DataView – динамическое представление данных таблицы-источника, то есть все изменения в таблице-источнике немедленно отображаются в объекте DataView. Каждый объект DataView является представлением только одной таблицы и не может быть объединением нескольких таблиц. Несмотря на то, что объект DataView очень похож на таблицу, он не может использоваться как таблица, в нем нельзя исключать поля, которые присутствуют в таблице-источнике, нельзя включать дополнительные поля, например, вычисляемые поля, которых нет в таблице-источнике. У каждой таблицы может быть сколько угодно представлений и для всех этих представлений таблица является источником полей и строк.

Первый способ создания объекта DataView – использовать свойство DefaultView непосредственно таблицы-источника.

ds.Tables["Salaries"].DefaultView.RowFilter=”Salary > 10000”;

ds.Tables["Emp"].DefaultView.Sort=”Department”;

Второй способ - явное создание объекта класса DataView для конкретной таблицы с последующей настройкой свойств RowFilter, Sort и RowStateFilter объекта.

DataView dv = new DataView(ds.Tables["Emp"], " ", "FIO",

DataViewRowState.CurrentRows);

Этой кодовой строкой было создано представление для таблицы Emp, в котором нет фильтра, сортировка осуществлена по полю FIO в возрастающем порядке и отображены текущие записи таблицы.

Объект DataView поддерживает модель редактирования, аналогичную модели редактирования объекта DataTable, то есть в представлении можно удалять, добавлять, редактировать данные. Создайте в вашем проекте еще одну форму для изучения объекта DataView (рисунок 6.1).

Произведите наполнение данными элементов интерфейса при загрузке формы как показано на рисунке 6.2.

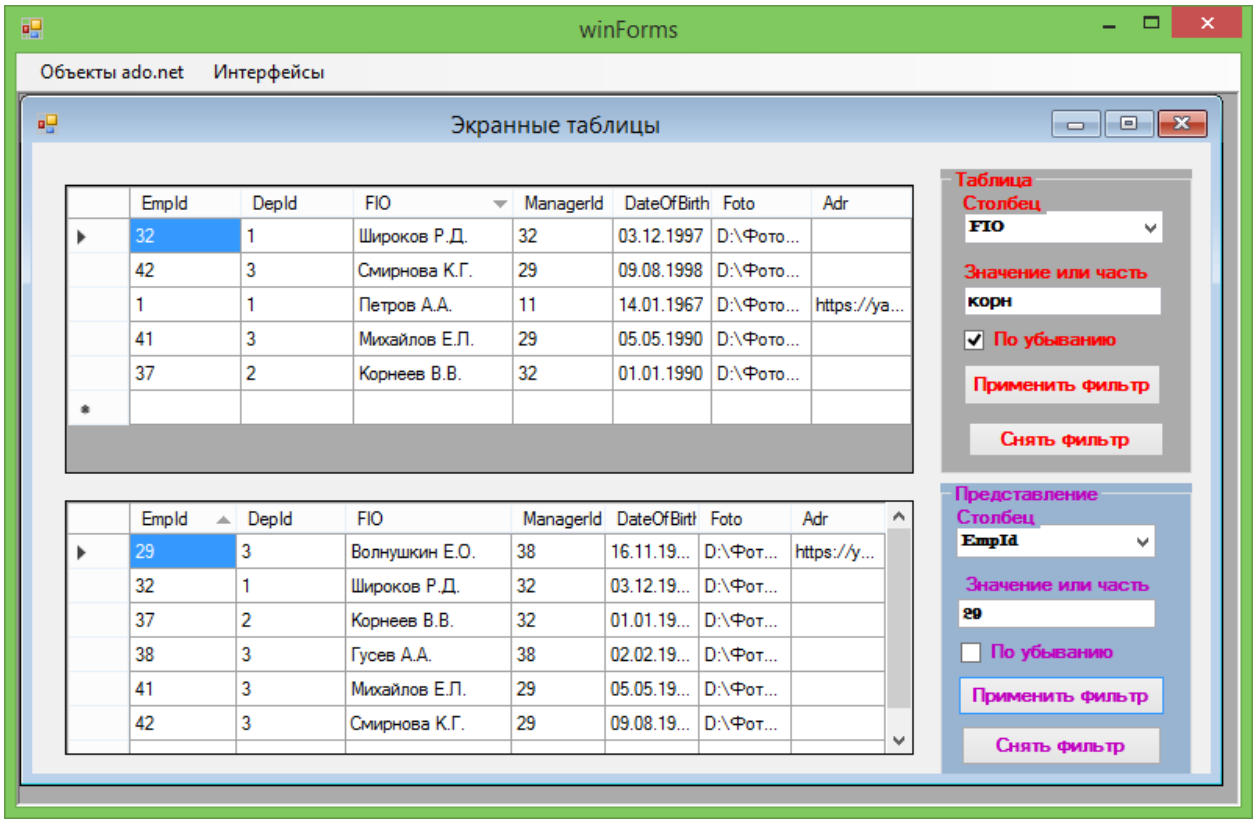


Рисунок 6.1 - Интерфейс, построенный для изучения объекта DataView

```
        private void Form8_Load(object sender, EventArgs e)
        {
cnn = new SqlConnection(ConfigurationManager.ConnectionStrings["Employees"].ConnectionString);
            da = new SqlDataAdapter("select * from Emp", cnn);
            da.Fill(ds, "Emp");
            dataGridView1.DataSource = ds.Tables["Emp"];

            //Создаем представление для таблицы Emp
            dv = new DataView(ds.Tables["Emp"],"","FIO",DataViewRowState.CurrentRows);

            //Чтение списка названий столбцов таблицы в comboBoxы
            foreach (DataColumn col in ds.Tables["Emp"].Columns)
            {
                comboBox1.Items.Add(col.ColumnName);
                comboBox2.Items.Add(col.ColumnName);
            }

            //Установка значений по умолчанию
            comboBox1.SelectedItem = "FIO";
            comboBox2.SelectedItem = "FIO";
            ds.Tables["Emp"].DefaultView.Sort = "FIO";
            dv.Sort = "FIO";

            //Связывание сеток с 2-мя различными представлениями таблицы
            dataGridView1.DataSource = ds.Tables["Emp"].DefaultView;
            dataGridView2.DataSource = dv;
        }
```

Рисунок 6.2 – Использование DataView для манипулирования с данными

Свойство Sort предназначено для вывода записей в порядке возрастания (ascending, ASC) или убывания (descending, DESC) по значениям заданного поля. В принципе, элемент DataGridView сам по себе поддерживает сортировку – достаточно просто щелкнуть по заголовку поля. Однако это требует действий от пользователя, тогда как объект DataView может предоставлять данные уже в готовом виде.

На рисунке 6.3 приведен программный код организации фильтрации в представлении по умолчанию.

```
    //Верхняя кнопка Применить фильтр
    private void button1_Click(object sender, EventArgs e)
    {
        //Указание фильтра
string filter = String.Format("{0}>='{1}'",comboBox1.SelectedItem.ToString(),textBox1.Text);
        ds.Tables["Emp"].DefaultView.RowFilter = filter;
        //Указание сортировки
        string sort = comboBox1.SelectedItem.ToString();
        if (checkBox1.Checked) sort += " DESC";
        ds.Tables["Emp"].DefaultView.Sort = sort;
    }
```

Рисунок 6.3 – Фильтрация данных в представлении по умолчанию

Свойство RowFilter позволяет формировать условие фильтрации записей. Это свойство, задаваемое строкой, применяется в качестве средства фильтрации на базе критериев, определенных значением строки. Ее синтаксис подобен конструкции WHERE стандартного SQL, но относится к данным, уже полученным из базы. DataView не позволяет фильтровать столбцы, только строки. На рисунке 6.4 программный код организации фильтра данных.

```
//Нижняя кнопка Применить фильтр
private void button3_Click(object sender, EventArgs e)
{
    //Указание фильтра
    string filter = String.Format("{0}>='{1}'", comboBox2.SelectedItem.ToString(),
      textBox2.Text);
    dv.RowFilter = filter;
    //Указание сортировки
    string sort = comboBox2.SelectedItem.ToString();
    if (checkBox2.Checked) sort += " DESC";
    dv.Sort = sort;
}
```

Рисунок 6.4 – Фильтрация данных

Рассмотрим практическую задачу построения интерфейса с использованием DataView. Например, требуется разработать отчет с вычислением всех выплаченных сумм в разрезе сотрудника. В отчете необходимо реализовать фильтрацию строк по значению одного из полей и сортировку для указанного столбца. Внешний вид отчета, построенного на объекте DataView показан на рисунке 6.5.

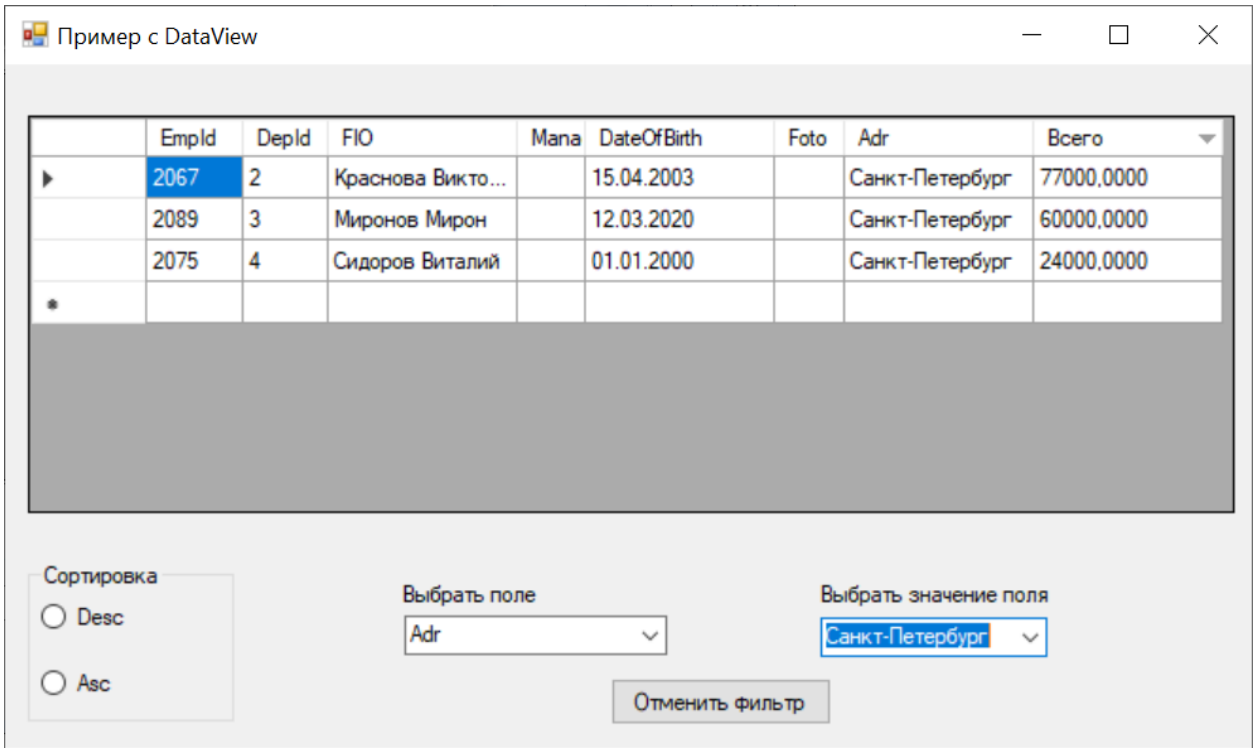


Рисунок 6.5 – Применение объекта DataView для организации просмотра данных

7. ПРОГРАММИРОВАНИЕ АДАПТЕРОВ ДАННЫХ

Объект ***DataAdapter*** конкретного поставщика данных автоматически обслуживает подключение к БД. Для повышения масштабируемости адаптеры данных держат подключение открытым минимально возможное время. Как только вызывающий процесс получит объект DataSet, вызывающий слой полностью отключается от БД и остается с локальной копией удаленных данных. Теперь в нем можно вставлять, удалять или изменять строки различных объектов DataTable, но физическая БД не обновляется, пока вызывающий процесс явно не передаст объект DataSet адаптеру данных для обновления. По сути, объекты DataSet имитируют постоянное подключение клиентов, хотя на самом деле они работают с находящейся в памяти БД. Объект DataAdapter отвечает за передачу любого обновления, вставки или удаления в физическую БД.

Данные из связанных таблиц базы данных при передаче их в приложение считываются отдельно в набор данных различными адаптерами.

Адаптер данных имеет четыре свойства: ***SelectCommand***, ***InsertCommand***, ***UpdateCommand*** и ***DeleteCommand***, но объект DataAdapter не создает автоматически команды INSERT, UPDATE и DELETE для обновления источника данных в соответствии с изменениями данных в объекте DataSet. Эти команды можно указать различными способами, например, следующими:

- Использовать объект ***CommandBuilder*** для автоматической генерации команд во время выполнения приложения.
- Явно запрограммировать эти команды.
- Использовать хранимые процедуры.

Рассмотрим технологию программирования адаптера данных для выполнения вставки, удаления, изменения строк таблицы.

Для таблиц Emp (Сотрудники) и Dep (Отделы) базы данных «Отдел кадров» построим интерфейс, в котором пользователь может заполнять таблицу Dep новыми записями, удалять и изменять имеющиеся. Сведения о

руководителе отдела, хранящиеся в таблице Emp, можно выбирать из списка, но не вводить новые и не удалять имеющиеся записи таблицы Emp.

На рисунке 7.1 приведен фрагмент модели базы данных с выделенным стрелкой отношением, которое формирует значения поля DepartmentHeadId (Табельный номер руководителя) для таблицы Dep.

На рисунке 7.2 показан пример интерфейса формы в режиме конструирования. Для представления связанных данных о руководителе отдела (поле DepartmentHeadId таблицы Dep) можно использовать элемент управления comboBox с комплексной привязкой к столбцам EmpId и FIO таблицы Emp.

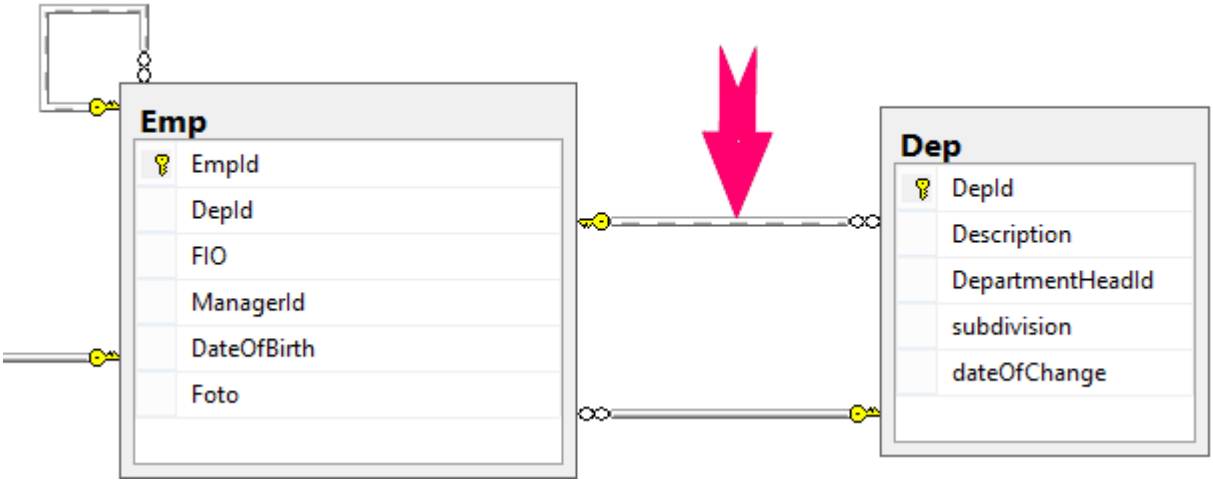


Рисунок 7.1 - Фрагмент модели

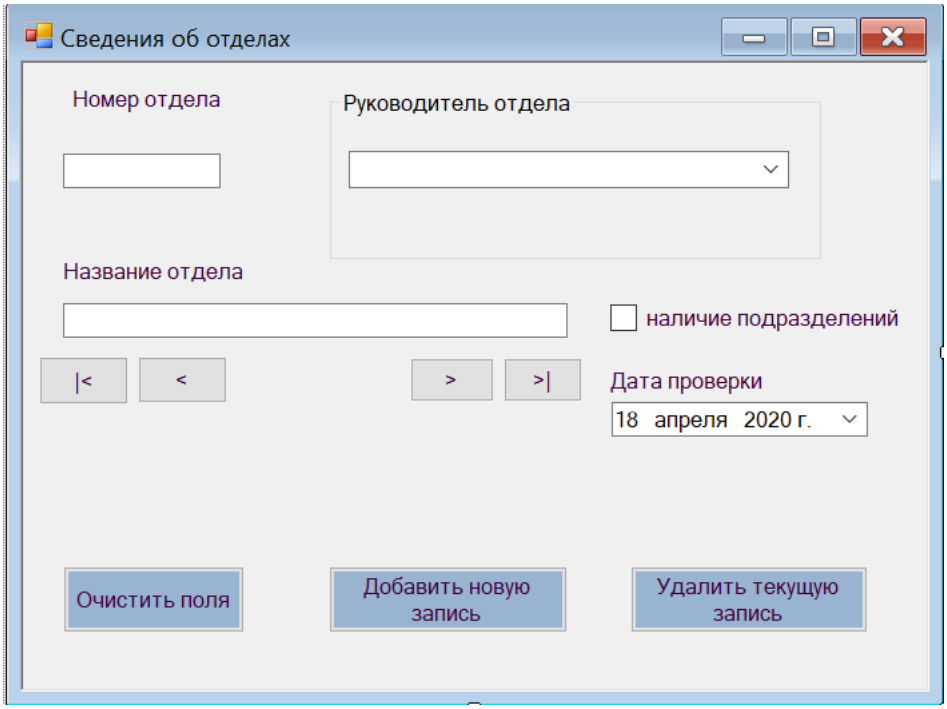


Рисунок 7.2 - Интерфейс формы в режиме конструктора

Программирование адаптеров для работы с таблицами Dep и Emp заключается в описании свойств SelectCommand, InsertCommand, UpdateCommand и DeleteCommand и показано на рисунке 7.3.

```
private void Form14_Load(object sender, EventArgs e)
{
    //Программируем адаптеры на выборку данных из таблиц базы данных
    daDep.SelectCommand=new SqlCommand("select * from Dep",cnn);
    daEmp.SelectCommand = new SqlCommand("select EmpId,FIO from Emp", cnn);
```

Рисунок 7.3 – Программирование свойства SelectCommand

Описание свойств адаптера таблицы Dep для команды вставки новых строк на рисунке 7.4.

```
//Программируем адаптер на вставку новых данных в таблицу Departments
daDep.InsertCommand = new SqlCommand("insert into Dep values (@DepId, @Description,
  @DepartmentHeadId, @sub, @datech)", cnn);
daDep.InsertCommand.Parameters.Add("@DepId",SqlDbType.Int,4,"DepId");
daDep.InsertCommand.Parameters.Add("@Description", SqlDbType.VarChar,50,"Description");
daDep.InsertCommand.Parameters.Add("@DepartmentHeadId", SqlDbType.Int,4, "DepartmentHeadId");
daDep.InsertCommand.Parameters.Add("@sub", SqlDbType.Bit, 1, "subdivision");
daDep.InsertCommand.Parameters.Add("@datech", SqlDbType.DateTime, 8, "dateOfChange");
```

Рисунок 7.4 – Программирование свойства InsertCommand

Изменение строк таблицы Dep показано на рисунке 7.5.

```
//Программируем адаптер на обновление данных в таблице Departments
daDep.UpdateCommand = new SqlCommand("update Dep set
  Description=@Description,DepartmentHeadId=@DepartmentHeadId,subdivision=@sub,
  dateOfChange=@datech  where DepId=@DepId", cnn);
daDep.UpdateCommand.Parameters.Add("@DepId", SqlDbType.Int,4, "DepId");
daDep.UpdateCommand.Parameters.Add("@Description", SqlDbType.VarChar, 50, "Description");
daDep.UpdateCommand.Parameters.Add("@DepartmentHeadId", SqlDbType.Int,4, "DepartmentHeadId");
daDep.UpdateCommand.Parameters.Add("@sub", SqlDbType.Bit, 1, "subdivision");
daDep.UpdateCommand.Parameters.Add("@datech", SqlDbType.DateTime, 8, "dateOfChange");
```

Рисунок 7.5 – Программирование свойства UpdateCommand

Удаление строк из таблицы Dep показано на рисунке 7.6.

```
//Программируем адаптер на удаление  данных из таблицы Departments
daDep.DeleteCommand = new SqlCommand("delete from Dep where DepId=@DepId", cnn);
daDep.DeleteCommand.Parameters.Add("@DepId", SqlDbType.Int,4, "DepId");
```

Рисунок 7.6 – Программирование свойства DeleteCommand

После получения выбранных данных из таблиц можно организовать интерфейс формы, например, с помощью объекта BindingSource как показано на рисунке 7.7.

```
//Заполняем хранилище данных
daDep.Fill(ds, "Dep");
daEmp.Fill(ds, "Emp");
//Создаем объект для синхронизации связанных данных
bind.DataSource = ds.Tables["Dep"];
```

Рисунок 7.7 – Синхронизация данных Binding Source

Просмотр и удаление текущей записи удобно выполнять свойствами и методами объекта BindingSource.

Добавление новой записи можно реализовать непосредственно в DataSet или через BindingSource.

Сохранение всех изменений, добавлений, удалений можно выполнить при закрытии формы.

8. ТЕХНОЛОГИЯ КОНСТРУИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

При конструировании программ необходимо обеспечить пользователя продуманной технологией ввода, редактирования и удаления данных.

Необходимо разработать приложение для кадрового учета и начисления заработной платы сотрудникам компании. Для реализации приложения будут использованы данные таблиц Сотрудники (Emp), Отделы (Dep), Зарплаты (Salaries), показанные на рисунке 8.1.

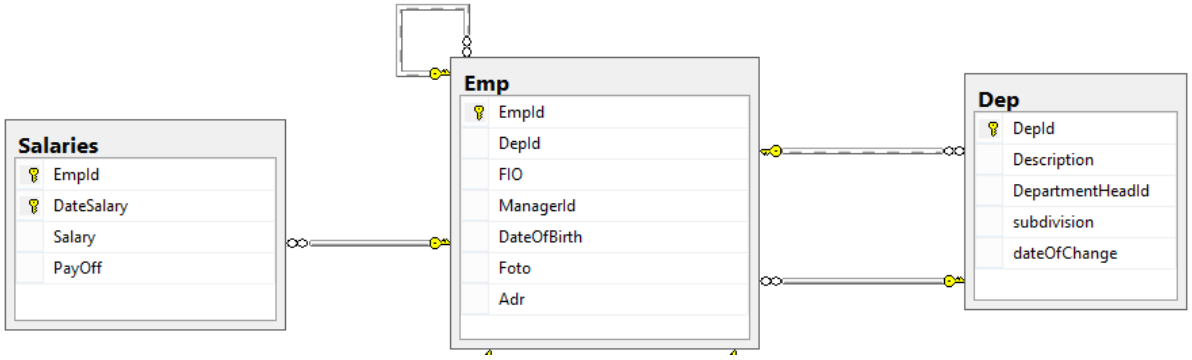


Рисунок 8.1 – Таблицы базы данных

Изменения данных будет обеспечено в таблицах Сотрудники (Emp) и Зарплаты (Salaries). Данные таблицы Отделы (Dep) будут использоваться как справочные. Для реализации поставленной задачи разработана форма рисунок 8.2.

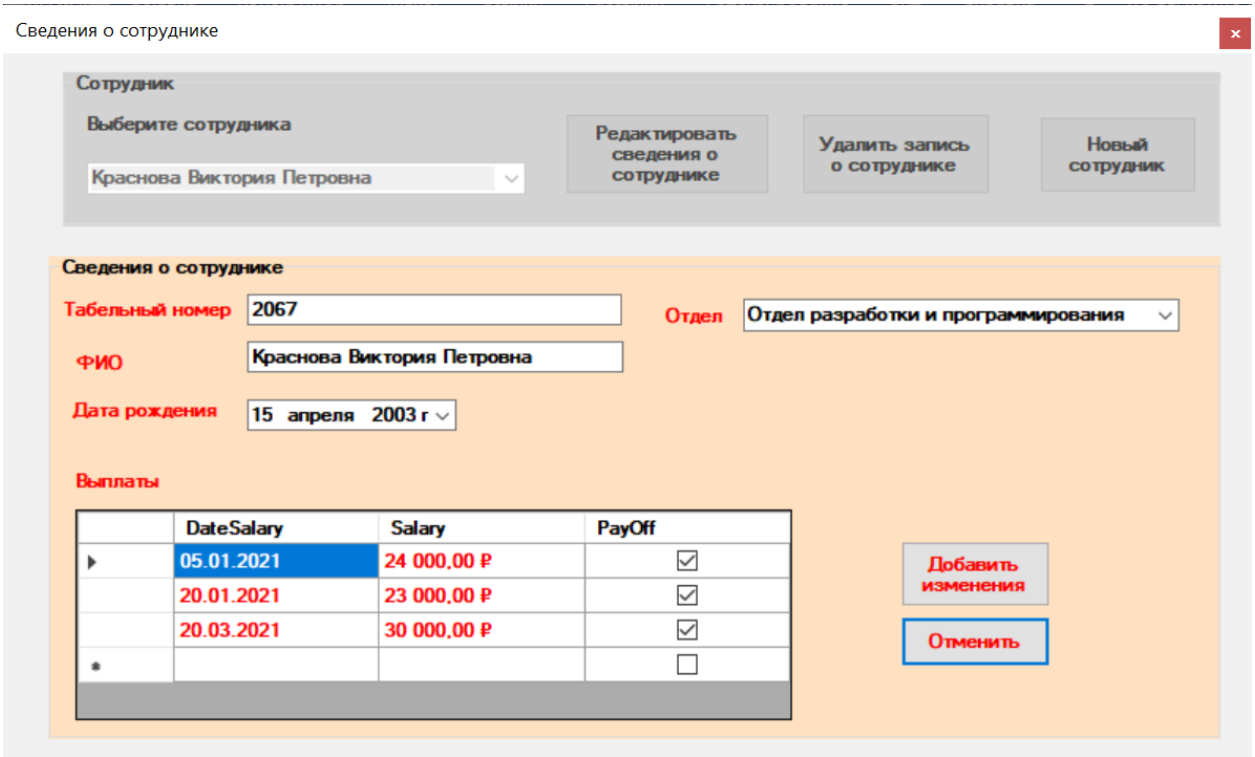


Рисунок 8.2 - Форма для редактирования информации о сотруднике

В соответствие с задачей данные из таблицы Emp в форме можно просматривать, удалять, редактировать, добавлять новые записи. Ключевой столбец таблицы EmpId имеет автозаполнение. Программирование адаптера данных для работы с таблицей Emp показано на рисунках 8.3 и 8.4.

```
        cnn = new SqlConnection(ConfigurationManager.ConnectionStrings
          ["Employees"].ConnectionString);
        //Команда SELECT для извлечения записей о сотрудниках
        daEmployees.SelectCommand = new SqlCommand("SELECT * from Emp",cnn);

    //Команда Insert для вставки новых строк в Employees
    daEmployees.InsertCommand = new SqlCommand("INSERT INTO Emp (DepId,FIO,
      DateOfBirth) values(@DepId,@FIO,@DateOfBirth)", cnn);
    daEmployees.InsertCommand.Parameters.Add("@DepId", SqlDbType.Int, 4,
      "DepId");
    daEmployees.InsertCommand.Parameters.Add("@FIO", SqlDbType.VarChar, 50,
      "FIO");
    daEmployees.InsertCommand.Parameters.Add("@DateOfBirth", SqlDbType.DateTime,
      8, "DateOfBirth");
```

Рисунок 8.3 – Программирование адаптера daEmployees на выборку и вставку

```
//Команда UPDATE для модификации существующих строк в таблице Employees
daEmployees.UpdateCommand = new SqlCommand("UPDATE Emp Set DepId=@DepId,
  FIO=@FIO,  DateOfBirth=@DateOfBirth WHERE EmpId=@EmpId",cnn);
daEmployees.UpdateCommand.Parameters.Add("@EmpId", SqlDbType.Int, 4,
  "EmpId");
daEmployees.UpdateCommand.Parameters.Add("@DepId", SqlDbType.Int, 4,
  "DepId");
daEmployees.UpdateCommand.Parameters.Add("@FIO", SqlDbType.VarChar, 50,
  "FIO");
daEmployees.UpdateCommand.Parameters.Add("@DateOfBirth", SqlDbType.DateTime,
  8, "DateOfBirth");

//Команда DELETE для удаления существующих строк из таблицы Employees
daEmployees.DeleteCommand = new SqlCommand("DELETE Salaries WHERE
  EmpId=@EmpId; Delete Emp WHERE EmpId=@EmpId",cnn);
daEmployees.DeleteCommand.Parameters.Add("@EmpId", SqlDbType.Int, 4,
  "EmpId");
```

Рисунок 8.4 - Программирование адаптера daEmployees на обновление и удаление

Данные из таблицы Salaries можно просматривать, редактировать, удалять и добавлять новые записи как показано на рисунке 8.5.

```
//Оператор SELECT для выборки данных таблицы Salaries
daSalary.SelectCommand = new SqlCommand("SELECT * from Salaries",cnn);
//Команда INSERT для вставки данных в таблицу Salaries
daSalary.InsertCommand = new SqlCommand("INSERT INTO Salaries VALUES
  (@EmpId,@DateSalary, @Salary, @PayOff)",cnn);
daSalary.InsertCommand.Parameters.Add ("@EmpId", SqlDbType.Int, 4, "EmpId");
daSalary.InsertCommand.Parameters.Add ("@DateSalary", SqlDbType.DateTime, 8,
  "DateSalary");
daSalary.InsertCommand.Parameters.Add ("@Salary", SqlDbType.Money, 8, "Salary");
daSalary.InsertCommand.Parameters.Add("@PayOff", SqlDbType.Bit, 1, "PayOff");
//Команда UPDATE для обновления информации в таблице Salaries
daSalary.UpdateCommand = new SqlCommand("UPDATE Salaries Set
  DateSalary=@NewDateSalary, Salary=@Salary, PayOff=@PayOff WHERE EmpID=@EmpId and
  DateSalary=@OldDateSalary",cnn);
daSalary.UpdateCommand.Parameters.Add ("@EmpId", SqlDbType.Int, 4, "EmpId");
daSalary.UpdateCommand.Parameters.Add ("@Salary", SqlDbType.Money, 8, "Salary");
daSalary.UpdateCommand.Parameters.Add ("@NewDateSalary", SqlDbType.DateTime, 8,
  "DateSalary");
daSalary.UpdateCommand.Parameters.Add ("@OldDateSalary", SqlDbType.DateTime, 8,
  "DateSalary");
daSalary.UpdateCommand.Parameters["@OldDateSalary"].SourceVersion =
  DataRowVersion.Original;
daSalary.UpdateCommand.Parameters
  ["@NewDateSalary"].SourceVersion=DataRowVersion.Current;
daSalary.UpdateCommand.Parameters.Add("@PayOff", SqlDbType.Bit, 1, "PayOff");
//Оператор DELETE, предназначенный для удаления данных из таблицы Salaries
daSalary.DeleteCommand = new SqlCommand("DELETE Salaries WHERE EmpId=@EmpId and
  DateSalary=@DateSalary",cnn);
daSalary.DeleteCommand.Parameters.Add ("@EmpId", SqlDbType.Int, 4, "EmpId");
daSalary.DeleteCommand.Parameters.Add ("@DateSalary", SqlDbType.DateTime, 8,
  "DateSalary");
```

Рисунок 8.5 – Программирование адаптера данных daSalary

Данные из таблицы Dep можно только просматривать.

При форматировании вывода данных в форме, например, в элементе DataGridView, можно использовать практические приемы, рассмотренные в документации Microsoft https://docs.microsoft.com/ru-ru/dotnet/desktop/winforms/controls/how-to-format-data-in-the-windows-forms-datagridview-control?view=netframeworkdesktop-4.8

Пример форматирования столбцов с датой и денежными суммами показан на рисунке 8.6.

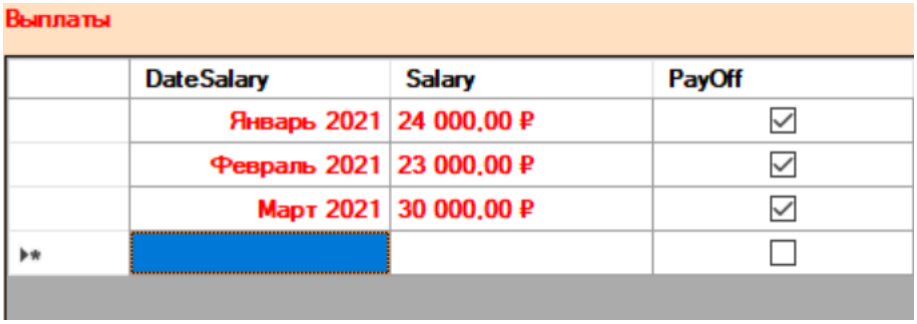


Рисунок 8.6 - Форматирование данных элемента управления DataGridView

9. ТЕХНОЛОГИИ ПЕРЕДАЧИ ДАННЫХ В ИНФОРМАЦИОННЫХ СИСТЕМАХ

Встроенные возможности DataSet позволяют производить некоторые операции с xml-документами, в частности, сохранять данные из DataSet в xml и, наоборот, загружать из xml в DataSet. Подобные возможности позволяют при необходимости использовать DataSet в качестве посредника между базой данных и xml -файлами.

9.1 Основные принципы записи данных в xml формат

Данные, хранящиеся в DataSet независимо от способа записи, и схемы этих данных, можно прочитать из DataSet и сохранить в xml -формате (с xml -схемой или без нее).

Для изучения методов работы с данными в xml-формате спроектируйте интерфейс формы, например, такой, как показано на рисунке 9.1. Форма позволяет отображать связанные записи из 2-х таблиц: Отделы и Сотрудники. На рисунке 9.2 показан результат работы с формой:

- чтение данных из DataSet в файл xml со схемой или без нее;
- чтение xml файла и запись в DataSet с отображением в dataGrid;
- чтение данных из базы данных в формате xml и отображение в richTextBox.

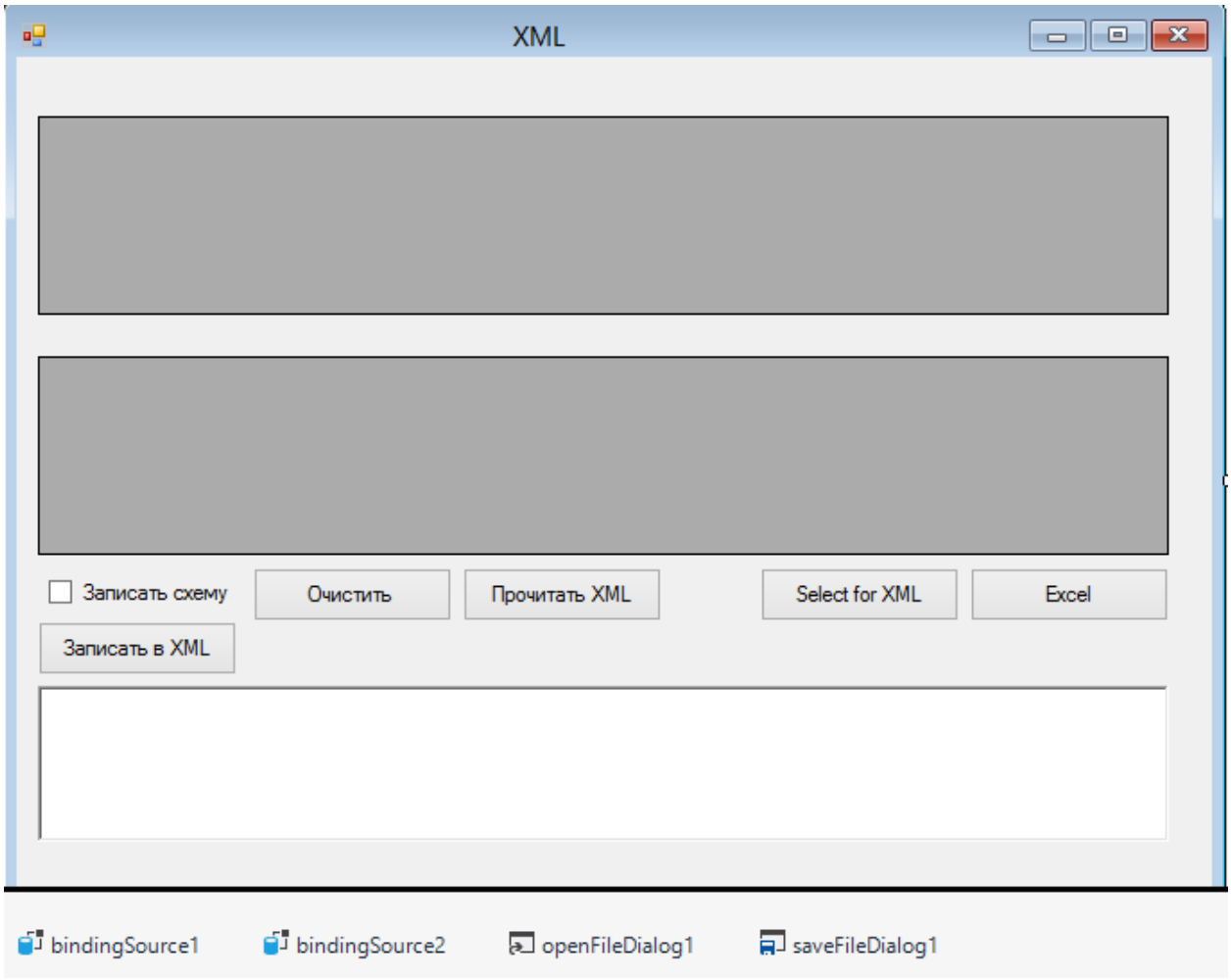


Рисунок 9.1 – Конструирование формы для исследования возможностей работы DataSet с данными в xml-формате

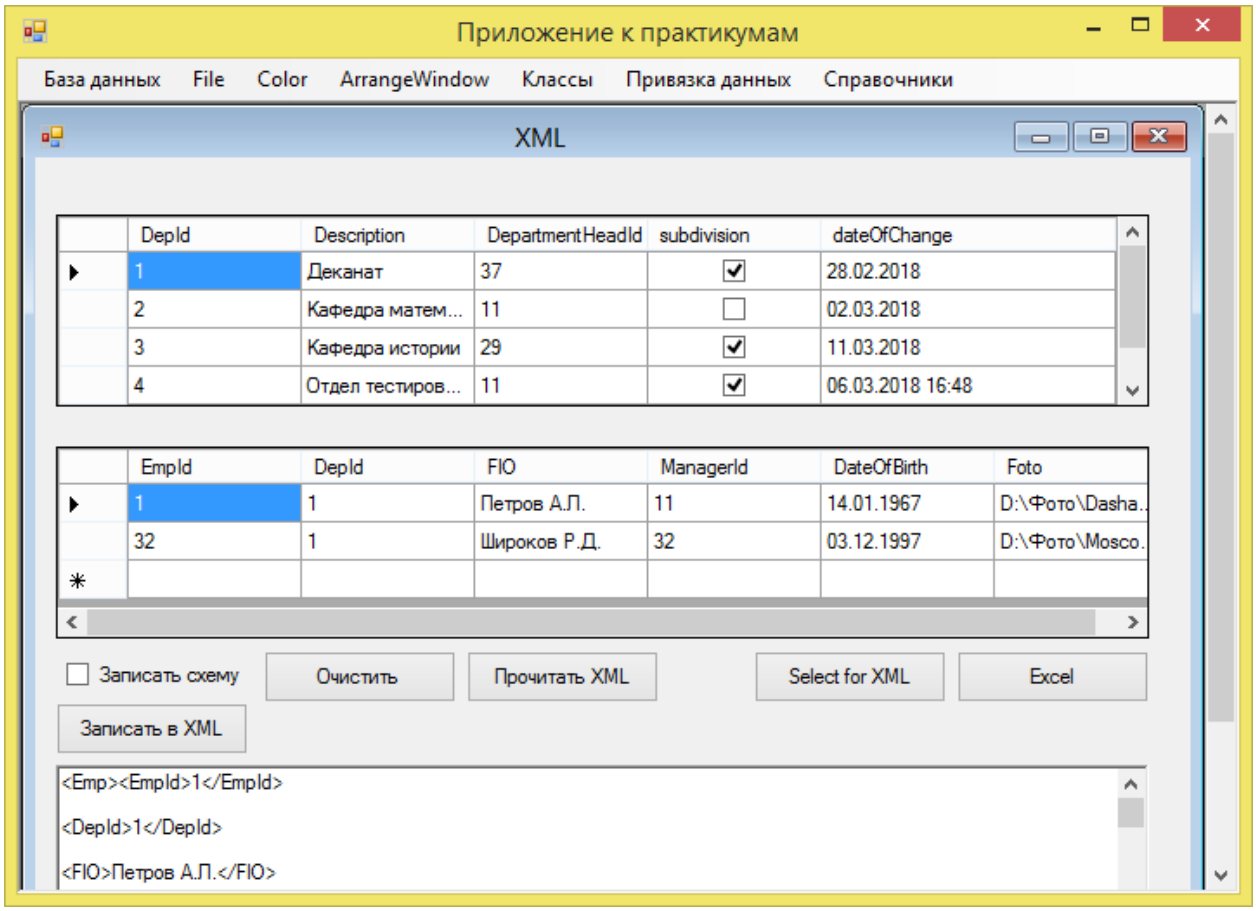


Рисунок 9.2 – Результаты чтения DataSet

Для записи реляционных данных в формат xml необходимо использовать методы объекта DataSet. Программный код на рисунке 9.3:

- метод WriteXml() позволяет преобразовать реляционные данные в xml формат,
- метод WriteXmlSchema() записывает схему отношений объектов, размещенных в DataSet.

```
//Запись объекта DataSet в файл xml
private void button1_Click(object sender, EventArgs e)
{
    saveFileDialog1.Title = "Запись файла в формате xml";
    if (saveFileDialog1.ShowDialog() != DialogResult.Cancel)
    {
        ds.WriteXml(saveFileDialog1.FileName+".xml");
    }
     if (checkBox1.Checked)   //Подтверждение записи схемы
    {
        saveFileDialog1.Title = "Запись схемы отношений xsd";

        if (saveFileDialog1.ShowDialog() != DialogResult.Cancel)
        {
        ds.WriteXmlSchema(saveFileDialog1.FileName+".xsd");
        }
    }
}
```

Рисунок 9.3 – Запись объекта DataSet в файл xml

На рисунке 9.4 показан файл Emp.xml, который создается в результате записи содержимого DataSet в xml -файл.

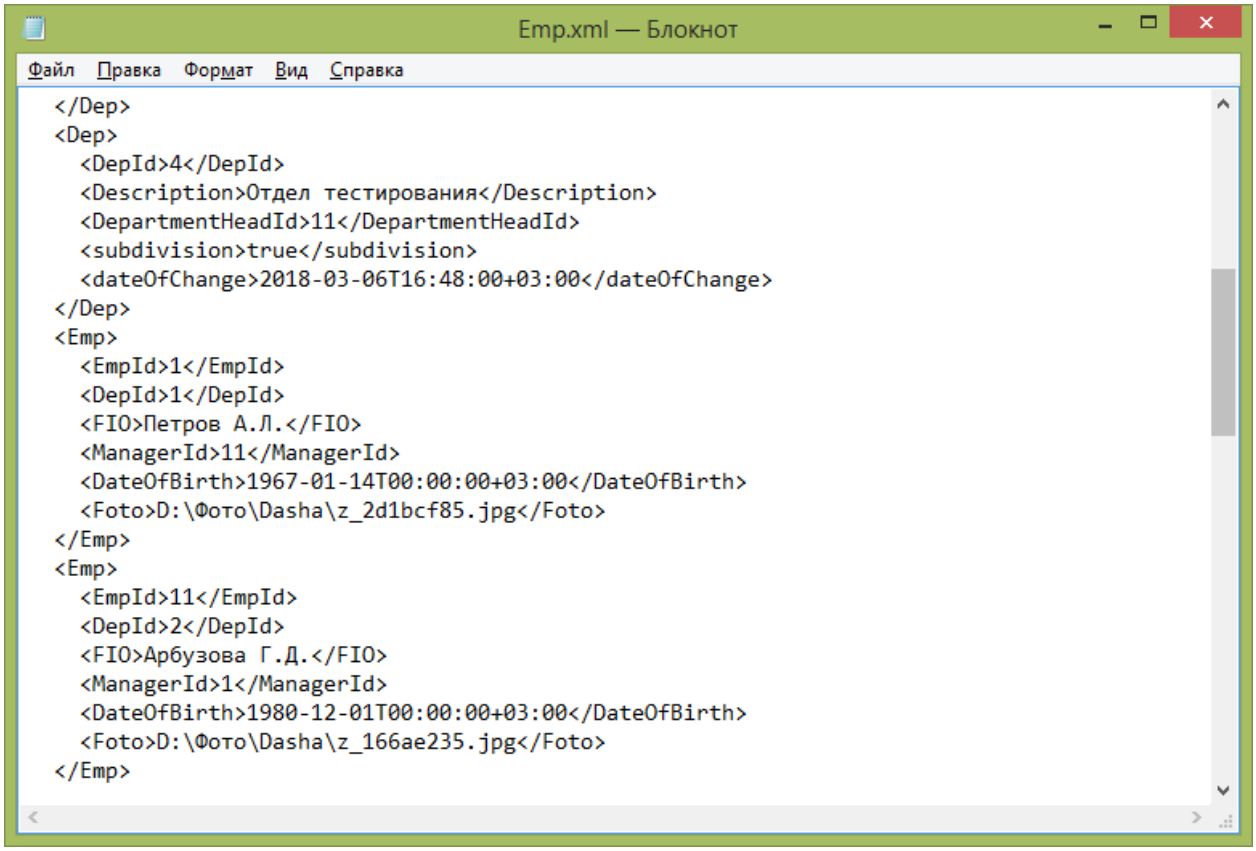


Рисунок 9.4 – Запись данных из DataSet в xml-файл

Обратите внимание, что xml-документ содержит записи об отделах, а ниже записи о сотрудниках несмотря на то, что между таблицами в DataSet

было установлено отношение, которое позволяет отображать связанные записи (рисунок 9.2).

Схема была сохранена в файл Emp.xsd. На рисунке 9.5 приведена схема двух связанных таблиц из файла Emp.xsd.

Для вложения дочерних записей из таблицы Emp в родительские строки таблицы Dep нужно указать значение true для свойства Nested объекта-отношения Relation, т.е. явно указать вложенность данных.

На рисунке 9.6 показан файл xml с вложением дочерних записей в соответствующие им родительские записи.

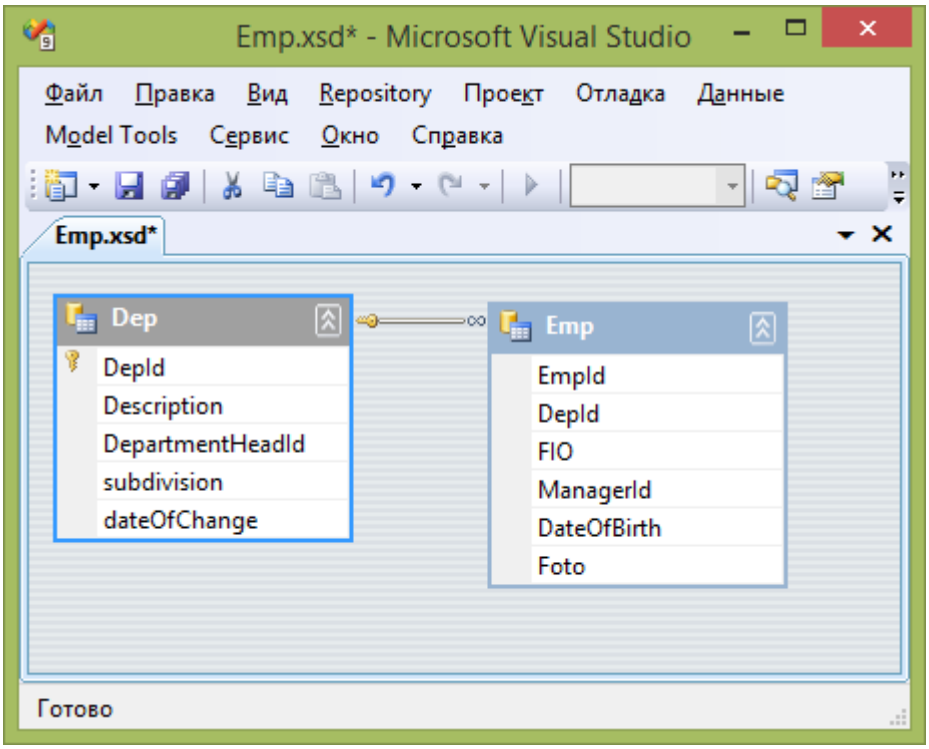


Рисунок 9.5 – Схема отношений между наборами данных

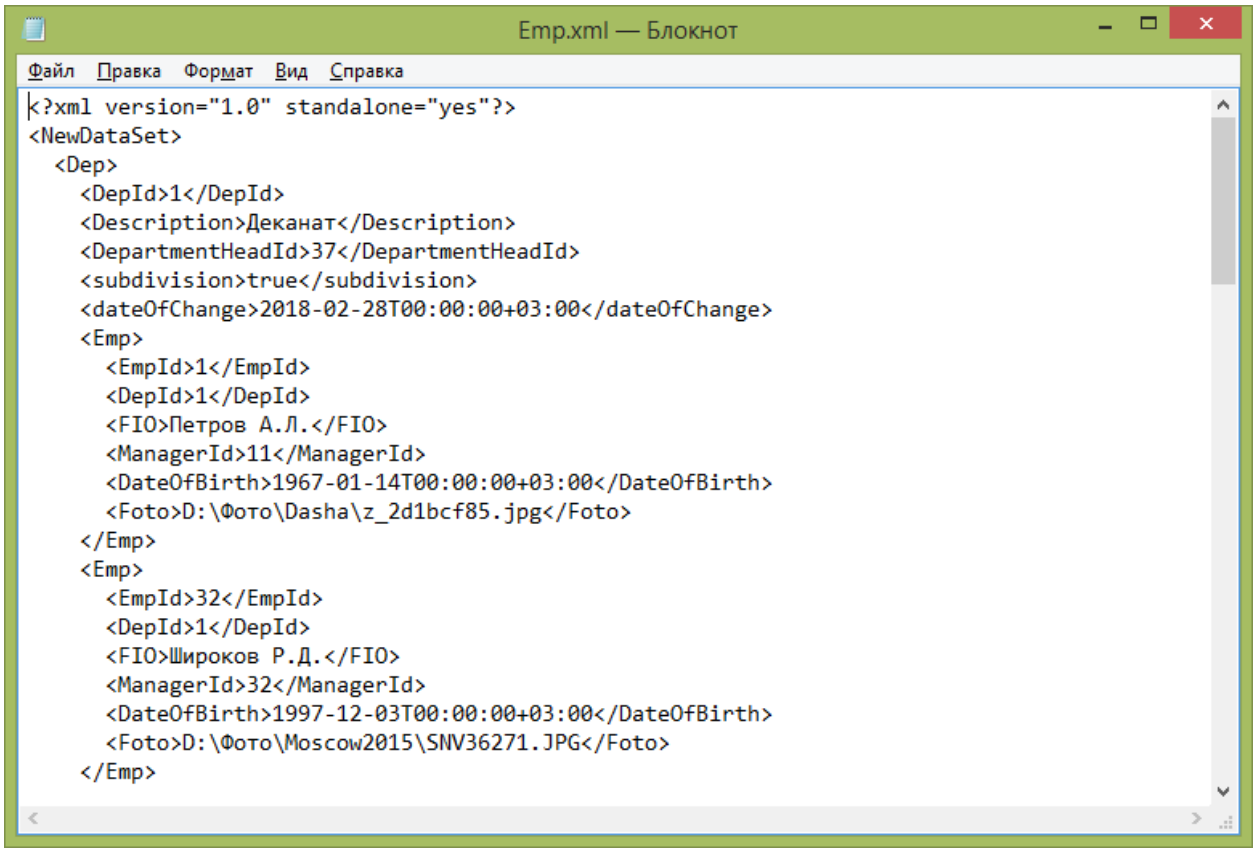


Рисунок 9.6 – Иерархия в данных

9.2 Извлечение и обработка данных из СУБД SQL Server в формате XML

Для извлечения и доступа к xml -данным из СУБД SQL Server можно использовать метод ExecuteXmlReader() объекта Command, который возвращает объект XmlReader.

Для демонстрации применения метода ExecuteXmlReader() выполните следующие действия:

- Добавьте в форму для задания 1 кнопку «Select For XML» и элемент richTextBox.
- Разберите и добавьте к обработчику события щелчка по кнопке следующий программный код рисунок 9.7:

```
//Выборка данных в формате XML из базы данных
private void button2_Click(object sender, EventArgs e)
{
    SqlCommand cmd = new SqlCommand("SELECT * FROM Employees FOR XML AUTO, elements", cnn);
    System.Xml.XmlReader reader;
    StringBuilder str = new StringBuilder();
    cnn.Open();
    //Выполнение SQL-команды Select с предложением FOR XML
    reader = cmd.ExecuteXmlReader();
    //Поиск и извлечение данных из узлов-элементов
    while (reader.Read())
    {
        switch (reader.NodeType)
        {
            case System.Xml.XmlNodeType.Element:
                str.Append("<" + reader.Name + ">");
                break;
            case System.Xml.XmlNodeType.EndElement:
                str.Append("</>" + reader.Name + ">" + "\n\r");
                break;
            case System.Xml.XmlNodeType.Text:
                str.Append(reader.Value);
                break;
            default:
                //Сейчас мы не будем рассматривать другие возможные XML-дескрипторы
                break;
        }
    }
    cnn.Close();
    richTextBox1.Text = str.ToString();

}
```

Рисунок 9.7 – Выборка данных в формате XML из базы данных

Предложенный к рассмотрению программный код является упрощенным примером использования метода ExecuteXmlReader(). Код примера позволяет просто отобразить данные (вместе с дескрипторами) из таблицы базы данных Emp. СУБД MS SQL Server передается SQL-команда SELECT, в которой явно указаны возвращаемые поля в виде XML-элементов:

"SELECT * FROM Emp FOR XML AUTO, elements"

В таком случае обработка разных типов xml -узлов упрощается, потому что достаточно найти только начальный и конечный узлы, а текстовые узлы между ними будут содержать фактические данные. Для случая полного разбора xml -документа необходимо учитывать узлы всех типов.

Подробнее про чтение реляционных данных в формат XML можно прочитать, например, в https://docs.microsoft.com/ru-ru/sql/relational-databases/xml/use-auto-mode-with-for-xml?view=sql-server-ver15

9.3 Экспорт данных в приложения MS Office

Элемент управления DataGridView исключительно полезен для приложений, работающих с базами данных, однако его функциональности не всегда достаточно. Часто данные необходимо экспортировать, например, в Microsoft Excel для дальнейшей обработки.

То, что MS Excel умеет отображать xml -документы, сильно упрощает задачу экспорта. Если загрузить в Excel файл xml, в котором сохранены данные из формы, можно получить следующие объекты: таблицу и карту xml. Попробуйте проделать для своих xml файлов.

Но можно предложить пользователю самостоятельно не открывать Excel, а выполнить все действия по экспорту из формы. Программный код на рисунке 9.8, результат выгрузки данных в Excel – рисунок 9.9.

```
//Необходимо добавить ссылку и пространство имен
using Excel = Microsoft.Office.Interop.Excel;

    //Выгрузить в Excel
    ссылка: 1
    private void button5_Click(object sender, EventArgs e)
    {
        Excel.Application excelApp = new Excel.Application();
        Excel.Workbook excelWorkBook= excelApp.Workbooks.Add();
        Excel.Worksheet excelWorkSheet = excelWorkBook.Worksheets.Item[1] as Excel.Worksheet;

        for (int i = 1; i < dataGridView1.Columns.Count + 1; i++)
        {
            excelWorkSheet.Cells[1, i] = dataGridView1.Columns[i - 1].HeaderText;
        }
        for (int i = 0; i < dataGridView1.Rows.Count - 1; i++)
        {
            for (int j = 0; j < dataGridView1.Columns.Count; j++)
            { excelWorkSheet.Cells[i + 2, j + 1] = dataGridView1.Rows[i].Cells
              [j].Value.ToString(); }
        }
        excelApp.Visible = true;
    }
```

Рисунок 9.8 – Выгрузка данных в Excel

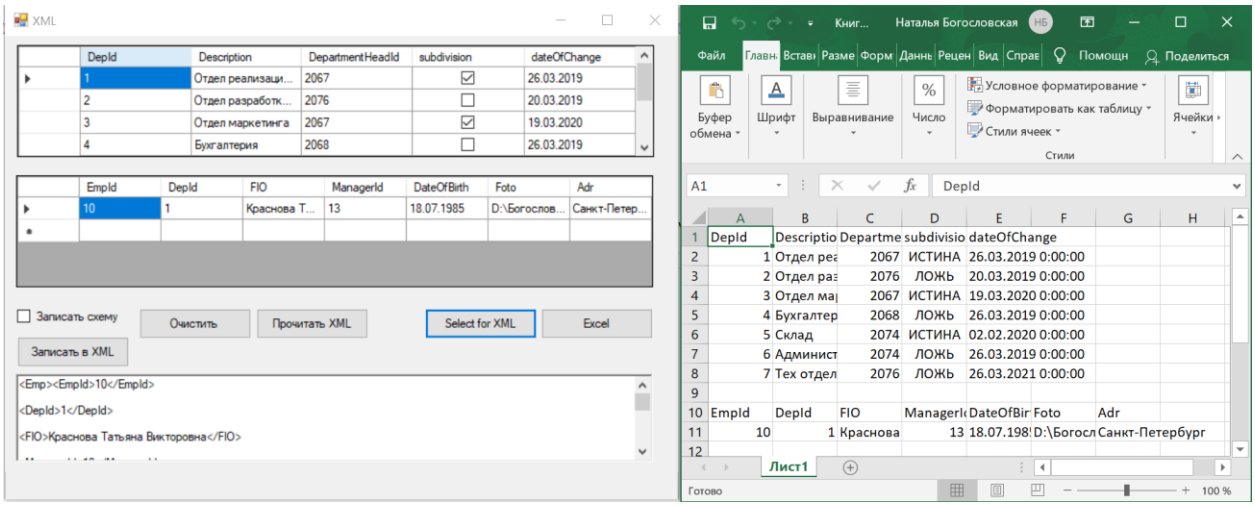


Рисунок 9.9 – Экспорт в Excel

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Вигерс К., Битти Дж. Разработка требований к программному обеспечению. 3-е изд., дополненное / Пер. с англ. — М. : Издательство «Русская редакция» ; СПб. : БХВ-Петербург, 2014. — 736 стр. : ил.
2. Peter Pin-Shan Chen. The entity-relationship model—toward a unified view of data. // ACM Transactions on Database Systems (TODS), 1976. — Vol. 1. — P. 9–36.
3. Microsoft Docs. Документация. ADO.NET // [Электронный ресурс] / Режим доступа: https://docs.microsoft.com/ru-ru/dotnet/framework/data/adonet/
4. Microsoft .NET. ASP.NET// [Электронный ресурс] / Режим доступа: https://dotnet.microsoft.com/apps/aspnet
5. Дейт К. Дж. Введение в системы баз данных. / пер. с англ. — 8-е изд. — М.: Вильямс, 2005. — 1138 с.
6. Чилберто Дж., Арораа Г. Паттерны проектирования для C# и платформы .NET Core / пер. с англ.; под ред. С.М. Молявко. — СПб: Питер, 2021. — 352 с.
7. Прайс Марк. C# 7 .NET Core. Кросс-платформенная разработка для профессионалов. /Пер. с англ. — 3-е изд. — СПб.: Питер, 2019. — 640с.
8. Албахари Дж., Албахари Б. C# 7.0 Карманный справочник. : пер. с англ. — СПб.: ООО «Альфа-книга», 2017. — 224 с.
9. Прайс М. C# 8 и .NET Core. Разработка и оптимизация. : пер. с анг. — СПб.: Питер, 2021. — 816 с.
10. Троелсон Э., Джепикс Ф. Язык программирования C# 7 и платформа .NET и .NET Core. /Пер. с англ. — М.: Вильямс, 2018. — 1328 с.

СОДЕРЖАНИЕ

1. ОСНОВНЫЕ ПОНЯТИЯ ТЕОРИИ ИНФОРМАЦИОННЫХ ПРОЦЕССОВ И СИСТЕМ 2
1.1 Централизация хранения данных в информационных системах 3
1.2 Разработка концептуальной модели в CASE-дизайнере 4
1.3 Технология разработки 8
2. ТЕХНОЛОГИИ ДОСТУПА К ДАННЫМ В ИНФОРМАЦИОННЫХ СИСТЕМАХ 11
2.1 Объект соединения 11
2.2 Управление строками соединений в windowsForms 12
2.3 Управление строками соединений в webForms 13
2.4 Основные объекты провайдера данных: объект Command 15
3. ХРАНИМЫЕ ПРОЦЕДУРЫ 21
3.1 Синтаксис параметризованных команд 21
3.2 Объект Command и вызов хранимых процедур 22
3.3 Транзакции и ADO.NET. Локальные транзакции 25
4. АВТОНОМНАЯ МОДЕЛЬ ДОСТУПА ADO.NET И КЛАСС DATASET 28
4.1 Объекты DataSet и DataAdapter 28
4.2 Программирование вставки данных в объекте DataTable 30
4.3 Синтаксис адресации полей DataSet и элементов DataGridView 33
5. АВТОНОМНАЯ МОДЕЛЬ ДОСТУПА К РЕЛЯЦИОННЫМ ДАННЫМ 35
5.1 Извлечение данных из форм, являющихся справочниками 35
5.2 Привязка данных. Простая привязка данных в формах Windows 38
5.3 Привязка данных к элементу DataGridView в Windows Forms 40
5.4 Привязка данных к элементу управления DetailsView в Web Form 41
5.5 Составная привязка данных в формах Windows 42
6. ПРЕДСТАВЛЕНИЕ ПОДМНОЖЕСТВА ДАННЫХ В ИНТЕРФЕЙСЕ ПОЛЬЗОВАТЕЛЯ 45
7. ПРОГРАММИРОВАНИЕ АДАПТЕРОВ ДАННЫХ 50
8. ТЕХНОЛОГИЯ КОНСТРУИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 54
9. ТЕХНОЛОГИИ ПЕРЕДАЧИ ДАННЫХ В ИНФОРМАЦИОННЫХ СИСТЕМАХ 58
9.1 Основные принципы записи данных в xml формат 58
9.2 Извлечение и обработка данных из СУБД SQL Server в формате XML 63
9.3 Экспорт данных в приложения MS Office 65
БИБЛИОГРАФИЧЕСКИЙ СПИСОК 67
ПРИЛОЖЕНИЕ 69

ПРИЛОЖЕНИЕ

Варианты заданий

№ п/п	Примерный перечень тем для самостоятельной работы
1	Информационная система учета реализации (продажи) товаров со склада
2	Информационная система кадрового учета организации
3	Информационная система учета договоров с контрагентами
4	Информационная система учета заказов клиентов
5	Информационная система бронирования отелей
6	Информационная система учета товаров в магазине
7	Информационная система учета студентов университета
8	Информационная система online бронирования билетов
9	Информационная система учет платежей
10	Информационная система записи к врачу в медицинский центр
11	Информационная система учет компьютерного оборудования в организации
12	Информационная система учет клиентов отдела кредитования банка
13	Информационная система учета номерного фонда гостиницы
14	Информационная система учета транспортных средств компании
15	Информационная система учета услуг туристического агентства
16	Информационная система учета клиентов и договоров агентства недвижимости
17	Информационная система учет договоров клиентов страховой компании
18	Информационная система учета услуг рекламного агентства
20	Информационная система